

1. Introdução

A transformação digital e o avanço da Indústria 4.0 vêm promovendo mudanças significativas na forma como as organizações coletam, armazenam e utilizam dados em seus processos produtivos. Nesse cenário, a informação passou a assumir papel estratégico dentro das empresas, contribuindo diretamente para atividades de planejamento, controle e tomada de decisão organizacional (Laudon; Laudon, 2014). Dessa forma, a Engenharia de Produção tornou-se cada vez mais dependente de dados confiáveis e atualizados para o gerenciamento eficiente de operações relacionadas à produção, logística, estoque, qualidade e desempenho operacional.

Os Sistemas de Informação Gerencial (SIG) possuem papel fundamental nesse contexto, uma vez que permitem transformar dados brutos em informações úteis para apoio à gestão organizacional (O'Brien; Marakas, 2013). Entretanto, em muitas organizações, os profissionais ainda dependem de relatórios prontos, planilhas exportadas e dashboards previamente configurados, o que pode limitar a flexibilidade das análises realizadas. Além disso, essas abordagens frequentemente apresentam problemas relacionados ao atraso na atualização das informações, excesso de agregação dos dados e dependência do setor de Tecnologia da Informação para execução de novas consultas e análises.

Diante disso, o acesso direto aos bancos de dados torna-se uma importante ferramenta de autonomia analítica para o engenheiro de produção, permitindo consultas mais específicas, maior rastreabilidade das informações e análises alinhadas às necessidades operacionais da organização. Segundo Elmasri e Navathe (2018), os bancos de dados são essenciais para garantir integridade, consistência e organização das informações dentro dos sistemas computacionais modernos, possibilitando maior confiabilidade no armazenamento e recuperação dos dados.

Além disso, o crescimento do volume e da variedade de dados industriais impulsionou o desenvolvimento de diferentes modelos de bancos de dados. Os bancos relacionais, como PostgreSQL e MySQL, utilizam tabelas estruturadas e relacionamentos definidos, sendo amplamente aplicados em sistemas corporativos devido à sua consistência transacional e integridade referencial (Date, 2004). Por outro lado, os bancos de dados não relacionais, também conhecidos como NoSQL, como o MongoDB, surgiram como alternativa para aplicações que demandam maior flexibilidade, escalabilidade e manipulação de dados semiestruturados ou não estruturados (Sadalage; Fowler, 2013).

Nesse contexto, compreender os fundamentos relacionados aos bancos de dados, à arquitetura de software e aos sistemas de informação tornou-se uma competência relevante para profissionais da Engenharia de Produção, especialmente em ambientes industriais cada vez mais orientados por dados. Assim, o presente trabalho tem como objetivo elaborar um Procedimento Operacional Padrão (POP), no formato de artigo científico, para documentar o processo de acesso, conexão, extração e análise de dados em bancos de dados aplicados à Engenharia de Produção. Para isso, serão abordados conceitos relacionados aos Sistemas de Informação Gerencial, modelagem de dados, normalização, relacionamentos, agregações e composições, além da descrição dos procedimentos de conexão utilizando bancos de dados relacionais e não relacionais.

2. Fundamentação teórica

2.1. Sistemas de informação gerencial (SIG)

Os Sistemas de Informação Gerencial (SIG) podem ser definidos como sistemas responsáveis pela coleta, processamento, armazenamento e distribuição de informações utilizadas no apoio à gestão organizacional e à tomada de decisão (Laudon; Laudon, 2014). Em ambientes empresariais cada vez mais competitivos e orientados por dados, os SIG tornaram-se ferramentas essenciais para integração das informações provenientes dos diversos setores das organizações, permitindo maior controle, agilidade e eficiência operacional.

Segundo O'Brien e Marakas (2013), os sistemas de informação possuem a função de transformar dados brutos em informações úteis para os gestores, auxiliando no monitoramento das operações, no controle organizacional e no planejamento estratégico das empresas. Nesse contexto, os dados representam registros isolados de eventos ou atividades, enquanto a informação corresponde ao resultado do tratamento e interpretação desses dados, tornando-os relevantes para análise organizacional.

A relação entre dados, informação e conhecimento possui grande relevância dentro da Engenharia de Produção, especialmente em ambientes industriais que dependem de decisões rápidas e fundamentadas. De acordo com Stair e Reynolds (2015), o conhecimento é gerado a partir da interpretação das informações obtidas pelos sistemas organizacionais, permitindo que os gestores compreendam padrões, identifiquem problemas e proponham melhorias nos processos produtivos.

Na Engenharia de Produção, os Sistemas de Informação Gerencial possuem papel fundamental no gerenciamento de atividades relacionadas à produção, logística, estoque, manutenção,

qualidade e cadeia de suprimentos. A utilização eficiente das informações permite que os profissionais acompanhem indicadores de desempenho, reduzam desperdícios e realizem tomadas de decisão de maneira mais assertiva, contribuindo diretamente para o aumento da competitividade organizacional (Slack; Brandon-Jones; Johnston, 2018).

Além disso, os Sistemas de Informação Gerencial estão diretamente relacionados ao pensamento sistêmico, abordagem que considera a organização como um conjunto integrado de processos interdependentes (Chiavenato, 2014). Sob essa perspectiva, as decisões tomadas em determinado setor podem impactar diversas áreas da empresa, tornando fundamental a integração das informações organizacionais para garantir maior alinhamento estratégico e operacional.

Com o avanço da transformação digital e da Indústria 4.0, os sistemas de informação passaram a lidar com volumes cada vez maiores de dados provenientes de sistemas corporativos, sensores industriais, dispositivos conectados e plataformas digitais. Segundo Kagermann, Wahlster e Helbig (2013), a Indústria 4.0 está diretamente associada à utilização intensiva de dados e tecnologias digitais para integração dos processos produtivos, aumentando a necessidade de sistemas capazes de armazenar, processar e disponibilizar informações em tempo real.

Nesse cenário, os bancos de dados assumem papel central na estrutura dos sistemas de informação, sendo responsáveis pelo armazenamento, organização e recuperação das informações utilizadas pelas organizações. Dessa forma, compreender o funcionamento dos SIG e sua relação com os bancos de dados tornou-se uma competência relevante para profissionais da Engenharia de Produção que atuam em ambientes industriais orientados por dados e análise operacional.

2.2 Arquitetura de software

A arquitetura de software pode ser compreendida como a estrutura responsável pela organização dos componentes de um sistema computacional, definindo como esses elementos se relacionam e interagem entre si (Sommerville, 2019). Em sistemas corporativos, a arquitetura possui papel fundamental na separação das responsabilidades do sistema, permitindo maior organização, manutenção, escalabilidade e integração entre os diferentes processos organizacionais.

Segundo Pressman e Maxim (2020), uma arquitetura de software bem estruturada contribui diretamente para a qualidade do sistema, facilitando a manutenção, reutilização de componentes e evolução das aplicações ao longo do tempo. Dessa forma, as organizações conseguem

desenvolver sistemas mais eficientes e alinhados às necessidades operacionais e estratégicas da empresa.

Entre os modelos mais utilizados no desenvolvimento de sistemas empresariais destaca-se a arquitetura em camadas, também conhecida como *layered architecture*. Nesse modelo, o sistema é dividido em diferentes níveis de responsabilidade, normalmente compostos pela camada de apresentação, camada de regras de negócio e camada de acesso a dados (Fowler, 2002). Essa separação permite que alterações realizadas em uma camada tenham menor impacto sobre as demais, aumentando a flexibilidade e a organização do sistema.

Outro modelo amplamente difundido é a arquitetura MVC (Model-View-Controller), utilizada para separar a lógica de negócio, a interface visual e o controle das aplicações (Freeman e Robson, 2004). Nesse padrão, o Model representa os dados e regras do sistema, a View corresponde à interface apresentada ao usuário e o Controller atua como intermediador das requisições realizadas na aplicação. Essa estrutura facilita o desenvolvimento e manutenção dos sistemas, além de melhorar a organização do código-fonte.

Além do MVC, abordagens mais modernas, como a Clean Architecture, também passaram a ser utilizadas em sistemas corporativos e aplicações industriais. De acordo com Martin (2018), a Clean Architecture propõe uma estrutura desacoplada, na qual as regras de negócio permanecem independentes de frameworks, bancos de dados e interfaces externas. Nesse modelo, o banco de dados deixa de ser o centro da aplicação e passa a atuar como uma camada de infraestrutura, permitindo maior flexibilidade tecnológica e facilidade de manutenção.

No contexto da Engenharia de Produção, a arquitetura de software possui importância significativa devido à necessidade de integração entre diferentes sistemas organizacionais, como ERP, sistemas de estoque, controle de produção, logística e qualidade. Segundo Laudon e Laudon (2014), a integração entre sistemas permite maior confiabilidade das informações organizacionais e favorece a tomada de decisão baseada em dados atualizados e consistentes.

Os bancos de dados estão diretamente inseridos nessa arquitetura, sendo responsáveis pelo armazenamento e recuperação das informações utilizadas pelos sistemas empresariais. Dessa forma, compreender como os sistemas são estruturados e como ocorre a comunicação entre aplicação e banco de dados torna-se essencial para profissionais que atuam com análise de dados e suporte à tomada de decisão organizacional.

2.3 Histórico e evolução dos bancos de dados

O desenvolvimento dos bancos de dados está diretamente relacionado à necessidade crescente das organizações em armazenar, organizar e recuperar grandes volumes de informações de maneira eficiente e confiável. Segundo Elmasri e Navathe (2018), os primeiros sistemas utilizados para armazenamento de dados surgiram na década de 1960, baseados em arquivos sequenciais e modelos hierárquicos, nos quais os dados eram organizados em estruturas rígidas e de difícil flexibilização.

Entre os primeiros modelos utilizados destaca-se o modelo hierárquico, implementado em sistemas como o Information Management System (IMS), desenvolvido pela IBM. Nesse modelo, os dados eram organizados em estruturas semelhantes a árvores, nas quais cada registro possuía um único relacionamento pai-filho (Date, 2004). Apesar de apresentar bom desempenho para aplicações específicas, o modelo hierárquico possuía limitações relacionadas à flexibilidade e à complexidade para realização de consultas mais elaboradas.

Na década de 1970, ocorreu uma das principais transformações na área de bancos de dados com a proposição do modelo relacional por Edgar F. Codd. Segundo Codd (1970), os dados poderiam ser organizados em tabelas compostas por linhas e colunas, permitindo maior independência entre os dados e as aplicações. Esse modelo tornou-se um marco na evolução dos sistemas de banco de dados, principalmente pela simplificação da modelagem e pela maior facilidade na realização de consultas e relacionamentos entre informações.

A consolidação do modelo relacional ocorreu principalmente entre as décadas de 1980 e 1990, período em que surgiram diversos Sistemas Gerenciadores de Banco de Dados (SGBD), como Oracle, PostgreSQL, MySQL e SQL Server. Nesse período, a linguagem SQL (Structured Query Language) tornou-se o principal padrão para manipulação e consulta de dados relacionais, permitindo operações como inserção, atualização, exclusão e recuperação de informações de forma padronizada (Silberschatz; Korth; Sudarshan, 2020).

Com o avanço da internet, do comércio eletrônico e das aplicações distribuídas nos anos 2000, os bancos de dados relacionais passaram a enfrentar desafios relacionados à escalabilidade, flexibilidade e processamento de grandes volumes de dados não estruturados. Nesse contexto, surgiram os bancos de dados NoSQL, desenvolvidos para atender aplicações que demandavam alta disponibilidade, escalabilidade horizontal e manipulação de diferentes formatos de dados (Sadalage; Fowler, 2013).

Os bancos NoSQL englobam diferentes modelos de armazenamento, como bancos documentais, chave-valor, orientados a colunas e orientados a grafos. Entre eles, o MongoDB destacou-se pela utilização de documentos no formato JSON/BSON, permitindo maior

flexibilidade estrutural em comparação aos bancos relacionais tradicionais (Banker, 2011). Esse tipo de banco tornou-se amplamente utilizado em aplicações relacionadas à internet das coisas (IoT), análise de logs, sensores industriais e sistemas que necessitam manipular dados semiestruturados.

Atualmente, muitas organizações adotam a estratégia conhecida como polyglot persistence, que consiste na utilização de diferentes tipos de bancos de dados de acordo com as necessidades específicas de cada aplicação (Fowler; Sadalage, 2012). Nesse modelo, bancos relacionais e não relacionais podem coexistir dentro da mesma organização, permitindo combinar consistência transacional, flexibilidade estrutural e escalabilidade operacional.

No contexto da Engenharia de Produção, a evolução dos bancos de dados contribuiu significativamente para o aumento da capacidade de armazenamento e análise das informações organizacionais. Sistemas produtivos modernos passaram a gerar grandes volumes de dados relacionados à produção, manutenção, estoque, logística e qualidade, tornando os bancos de dados elementos essenciais para suporte à tomada de decisão e gestão eficiente das operações industriais

2.4 Bancos de dados relacionais e não relacionais

Os bancos de dados podem ser classificados em diferentes modelos de acordo com a forma como os dados são estruturados, armazenados e recuperados. Entre os principais modelos utilizados atualmente destacam-se os bancos de dados relacionais, baseados em tabelas estruturadas, e os bancos de dados não relacionais (NoSQL), desenvolvidos para aplicações que demandam maior flexibilidade e escalabilidade (Elmasri; Navathe, 2018).

Os bancos de dados relacionais foram desenvolvidos com base no modelo relacional proposto por Edgar F. Codd na década de 1970. Nesse modelo, os dados são organizados em tabelas compostas por linhas e colunas, permitindo o estabelecimento de relacionamentos entre diferentes entidades por meio de chaves primárias e chaves estrangeiras (Date, 2004). Essa estrutura favorece a integridade, a consistência e a padronização das informações armazenadas. Segundo Silberschatz, Korth e Sudarshan (2020), os bancos relacionais utilizam a linguagem SQL (Structured Query Language) como padrão para manipulação e consulta de dados. A linguagem SQL permite realizar operações de inserção, atualização, exclusão e recuperação de informações, além de possibilitar a utilização de JOINS para relacionar tabelas distintas dentro de uma mesma consulta.

Outra característica importante dos bancos relacionais está relacionada ao modelo transacional ACID (Atomicidade, Consistência, Isolamento e Durabilidade), responsável por garantir maior confiabilidade nas operações realizadas no banco de dados (Connolly; Begg, 2015). Essa característica torna os bancos relacionais amplamente utilizados em sistemas corporativos que exigem elevada integridade das informações, como sistemas ERP, controle financeiro, estoque e planejamento da produção.

Entre os principais bancos de dados relacionais utilizados atualmente destacam-se PostgreSQL, MySQL, Oracle Database e Microsoft SQL Server. O PostgreSQL, por exemplo, é amplamente reconhecido pela robustez, estabilidade e suporte a consultas complexas, sendo utilizado em aplicações empresariais e sistemas que demandam alta confiabilidade no armazenamento dos dados (PostgreSQL Global Development Group, 2025).

Por outro lado, os bancos de dados não relacionais, também conhecidos como NoSQL, surgiram como alternativa aos modelos relacionais tradicionais diante do crescimento exponencial do volume de dados gerados pelas aplicações modernas. Segundo Sadalage e Fowler (2013), os bancos NoSQL foram desenvolvidos para oferecer maior escalabilidade horizontal, flexibilidade estrutural e desempenho em aplicações distribuídas.

Diferentemente dos bancos relacionais, os bancos NoSQL não dependem necessariamente de tabelas rígidas e esquemas previamente definidos. No modelo documental, por exemplo, os dados são armazenados em documentos no formato JSON ou BSON, permitindo estruturas mais flexíveis e adaptáveis às necessidades das aplicações (Banker, 2011). Essa abordagem facilita o armazenamento de dados semiestruturados e reduz a necessidade de constantes alterações estruturais no banco de dados.

Entre os principais bancos de dados NoSQL destaca-se o MongoDB, amplamente utilizado em aplicações web, sistemas de monitoramento, internet das coisas (IoT), sensores industriais e armazenamento de logs operacionais. Segundo Chodorow (2019), o MongoDB oferece elevada flexibilidade para manipulação de grandes volumes de dados e permite expansão horizontal por meio da distribuição das informações entre diferentes servidores.

Além das diferenças estruturais, bancos relacionais e não relacionais também apresentam diferenças em relação ao modelo de consistência. Enquanto os bancos relacionais priorizam consistência forte por meio das propriedades ACID, muitos bancos NoSQL utilizam o modelo BASE (Basically Available, Soft State, Eventually Consistent), priorizando disponibilidade e escalabilidade em ambientes distribuídos (Brewer, 2012).

No contexto da Engenharia de Produção, ambos os modelos possuem aplicações relevantes. Bancos relacionais são amplamente utilizados em sistemas ERP, controle de estoque, planejamento e controle da produção, devido à necessidade de integridade e rastreabilidade das informações. Já bancos NoSQL apresentam vantagens em aplicações relacionadas à coleta de dados em tempo real, sensores industriais, monitoramento de máquinas e análise de grandes volumes de dados operacionais.

Dessa forma, a escolha entre bancos relacionais e não relacionais depende diretamente das características da aplicação, do volume de dados, da necessidade de escalabilidade e do tipo de informação manipulada. Atualmente, muitas organizações adotam arquiteturas híbridas, utilizando diferentes tecnologias de bancos de dados de forma complementar para atender demandas específicas dos sistemas organizacionais.

2.5 Normalização de dados e relacionamentos

A normalização de dados consiste em um processo de organização das informações em bancos de dados relacionais com o objetivo de reduzir redundâncias, evitar inconsistências e garantir maior integridade dos dados armazenados (Elmasri; Navathe, 2018). Segundo Date (2004), a normalização contribui para melhorar a eficiência estrutural do banco de dados, facilitando operações de inserção, atualização e remoção de informações.

De acordo com Silberschatz, Korth e Sudarshan (2020), o processo de normalização é dividido em diferentes etapas conhecidas como formas normais, sendo as três primeiras formas normais as mais utilizadas em sistemas corporativos. Essas etapas possuem como finalidade organizar adequadamente os atributos das tabelas e os relacionamentos existentes entre as entidades do banco de dados.

2.5.1 Primeira forma normal (1FN)

A Primeira Forma Normal (1FN) estabelece que cada atributo da tabela deve possuir apenas valores atômicos, ou seja, indivisíveis, eliminando listas, grupos repetitivos ou múltiplos valores em uma mesma célula (Connolly; Begg, 2015). Em ambientes industriais, um exemplo de violação da 1FN ocorre quando uma tabela de ordens de produção armazena diversos códigos de produtos em um único campo, dificultando consultas e manipulação dos dados.

2.5.2 Segunda forma normal (2FN)

A Segunda Forma Normal (2FN) determina que todos os atributos não pertencentes à chave primária devem depender totalmente da chave primária completa, eliminando dependências parciais (Date, 2004). Esse problema normalmente ocorre em tabelas que utilizam chaves compostas, nas quais determinados atributos dependem apenas de parte da chave principal. A aplicação da 2FN contribui para redução de redundâncias e inconsistências nos dados armazenados.

2.5.3 Terceira forma normal (3FN)

Já a Terceira Forma Normal (3FN) estabelece que atributos não pertencentes à chave primária não devem depender de outros atributos não chave, eliminando dependências transitivas (Elmasri; Navathe, 2018). Em um cenário industrial, isso pode ocorrer quando informações de fornecedores são armazenadas diretamente em tabelas de produtos, gerando repetição desnecessária de dados e maior risco de inconsistências durante atualizações.

Além da normalização, os relacionamentos entre tabelas representam elemento fundamental na modelagem de bancos de dados relacionais. Segundo Coronel e Morris (2016), os relacionamentos permitem representar associações existentes entre diferentes entidades organizacionais, contribuindo para maior integridade e organização das informações armazenadas.

2.5.4 Relacionamento 1:1

O relacionamento do tipo um para um (1:1) ocorre quando um registro de uma tabela está associado exclusivamente a um único registro de outra tabela. Um exemplo aplicado à Engenharia de Produção seria a relação entre funcionário e crachá, em que cada colaborador possui apenas um crachá vinculado ao seu cadastro (Silberschatz, Korth e Sudarshan, 2020).

2.5.5 Relacionamento 1:N

O relacionamento um para muitos (1:N) ocorre quando um registro de uma entidade pode estar relacionado a vários registros de outra entidade. Em sistemas industriais, um setor pode possuir diversos funcionários associados, enquanto cada funcionário pertence a apenas um setor específico (Connolly e Begg, 2015). Esse tipo de relacionamento é amplamente utilizado em sistemas ERP e bancos de dados corporativos.

2.5.6 Relacionamento N:M

Já o relacionamento muitos para muitos (N:M) ocorre quando múltiplos registros de uma entidade podem se relacionar com múltiplos registros de outra entidade. Segundo Elmasri e Navathe (2018), esse tipo de relacionamento normalmente exige a criação de uma tabela associativa intermediária. Um exemplo comum na Engenharia de Produção corresponde à relação entre produtos e fornecedores, na qual um fornecedor pode fornecer diversos produtos e um mesmo produto pode possuir múltiplos fornecedores cadastrados.

A correta aplicação das formas normais e dos relacionamentos contribui significativamente para a qualidade estrutural do banco de dados, favorecendo a integridade, rastreabilidade e confiabilidade das informações organizacionais. Dessa forma, a modelagem adequada dos dados torna-se essencial para garantir maior eficiência nas análises realizadas pelos sistemas de informação utilizados nas organizações.

2.6 Agregações e composições na modelagem de dados

A modelagem de dados possui papel fundamental na organização das informações em sistemas computacionais, permitindo representar entidades, atributos e relacionamentos de maneira estruturada e coerente com os processos organizacionais (Elmasri; Navathe, 2018). Nesse contexto, os conceitos de agregação e composição são utilizados para representar diferentes níveis de dependência entre os elementos existentes em um sistema.

Segundo Booch, Rumbaugh e Jacobson (2005), a agregação representa uma relação do tipo “tem um”, na qual os objetos associados possuem existência independente entre si. Nesse modelo, mesmo que a entidade principal deixe de existir, os elementos relacionados continuam existindo de forma autônoma dentro do sistema.

Na Engenharia de Produção, um exemplo de agregação pode ser observado na relação entre fornecedores e pedidos de compra. Embora um pedido esteja associado a determinado fornecedor, o fornecedor continua existindo no banco de dados mesmo que o pedido seja removido ou cancelado. Esse tipo de modelagem é amplamente utilizado em sistemas ERP e aplicações corporativas que trabalham com entidades independentes (Pressman; Maxim, 2020). Em bancos de dados relacionais, a agregação normalmente é implementada por meio de relacionamentos entre tabelas utilizando chaves estrangeiras e operações de JOIN para recuperação das informações relacionadas (Silberschatz; Korth; Sudarshan, 2020). Já em bancos de dados NoSQL, como o MongoDB, esse relacionamento pode ocorrer por meio de referências entre documentos utilizando identificadores do tipo ObjectId (Chodorow, 2019).

Por outro lado, a composição representa uma relação de dependência forte entre os elementos, caracterizando uma estrutura do tipo “parte de” (Booch; Rumbaugh; Jacobson, 2005). Nesse modelo, os elementos subordinados não possuem existência independente da entidade principal, sendo diretamente dependentes dela para existir dentro do sistema.

Um exemplo aplicado à Engenharia de Produção pode ser observado na relação entre ordens de produção e seus respectivos itens. Os itens pertencem exclusivamente à ordem de produção e deixam de existir caso a ordem seja removida do sistema. Esse tipo de modelagem é importante para garantir consistência e integridade das informações relacionadas aos processos produtivos (Slack; Brandon-Jones; Johnston, 2018).

Em bancos de dados relacionais, a composição geralmente é implementada utilizando tabelas filhas associadas por chaves estrangeiras com regras de exclusão em cascata, como o comando ON DELETE CASCADE (Connolly; Begg, 2015). Dessa forma, ao excluir o registro principal, todos os registros dependentes também são removidos automaticamente, preservando a integridade referencial do banco de dados.

Nos bancos de dados NoSQL, especialmente em bancos documentais como o MongoDB, a composição costuma ser representada por documentos embutidos (embedded documents), nos quais as informações subordinadas são armazenadas dentro do próprio documento principal (Banker, 2011). Essa abordagem favorece desempenho e simplifica consultas em cenários onde os dados possuem forte dependência estrutural.

A compreensão dos conceitos de agregação e composição é importante para a correta modelagem dos bancos de dados utilizados nas organizações, permitindo estruturar adequadamente os relacionamentos entre entidades e garantir maior eficiência no armazenamento e recuperação das informações. Além disso, esses conceitos contribuem para desenvolvimento de sistemas mais organizados, consistentes e alinhados às necessidades operacionais da Engenharia de Produção

3. Metodologia

3.1 Caracterização da pesquisa

O presente trabalho caracteriza-se como uma pesquisa de natureza aplicada, uma vez que busca utilizar conhecimentos teóricos relacionados a sistemas de informação, arquitetura de software e bancos de dados para elaboração de um Procedimento Operacional Padrão (POP) voltado ao contexto da Engenharia de Produção. O objetivo principal consiste em documentar o processo

de acesso, conexão, extração e análise de dados diretamente em bancos de dados utilizados em ambientes organizacionais.

Quanto aos objetivos, a pesquisa possui caráter exploratório e descritivo. Segundo Prodanov e Freitas (2013), pesquisas exploratórias possuem como finalidade proporcionar maior familiaridade com determinado problema, enquanto pesquisas descritivas buscam registrar, analisar e descrever características de fenômenos ou processos. Nesse contexto, o trabalho explora os conceitos relacionados aos bancos de dados e descreve os procedimentos necessários para realização das conexões e extração de informações.

A abordagem metodológica adotada possui caráter qualitativo e quantitativo. A abordagem qualitativa está relacionada à interpretação dos conceitos teóricos, modelagem de dados e análise dos procedimentos utilizados nos sistemas de informação. Já a abordagem quantitativa está associada à manipulação e análise de dados extraídos dos bancos de dados por meio de consultas estruturadas e técnicas computacionais.

Além disso, o trabalho foi estruturado no formato de artigo científico associado a um Procedimento Operacional Padrão (POP), conforme proposta apresentada na disciplina. Segundo Campos (1992), o POP possui como finalidade padronizar procedimentos operacionais, garantindo repetibilidade, organização e confiabilidade na execução das atividades descritas. Dessa forma, o presente artigo busca documentar de maneira clara e reproduzível os procedimentos relacionados ao acesso e análise de dados em bancos de dados aplicados à Engenharia de Produção.

A pesquisa utilizará como base bancos de dados relacionais e não relacionais disponibilizados para a disciplina, permitindo a análise comparativa entre diferentes modelos de armazenamento de dados. Para isso, serão abordados procedimentos de conexão utilizando PostgreSQL e MongoDB, além da utilização de ferramentas e bibliotecas computacionais voltadas à extração e manipulação das informações.

3.2 Tecnologias e ferramentas utilizadas

Para desenvolvimento do presente trabalho serão utilizados bancos de dados relacionais e não relacionais, possibilitando a aplicação prática dos conceitos abordados ao longo da fundamentação teórica. Segundo Elmasri e Navathe (2018), a utilização de diferentes modelos de bancos de dados permite compreender características específicas relacionadas à estrutura, escalabilidade e organização das informações.

O banco de dados relacional utilizado será o PostgreSQL, Sistema Gerenciador de Banco de Dados (SGBD) amplamente reconhecido pela robustez, confiabilidade e suporte a operações transacionais complexas (PostgreSQL Global Development Group, 2025). O PostgreSQL será utilizado para representar ambientes estruturados que exigem integridade referencial, normalização e relacionamento entre tabelas.

Para o contexto de banco de dados não relacional será utilizado o MongoDB, banco documental orientado a documentos JSON/BSON, amplamente utilizado em aplicações que demandam flexibilidade estrutural e escalabilidade horizontal (Chodorow, 2019). O MongoDB permitirá explorar conceitos relacionados ao armazenamento de dados semiestruturados e utilização de coleções documentais.

As conexões e manipulações dos bancos de dados serão realizadas utilizando a linguagem Python, devido à sua ampla utilização em aplicações voltadas para análise de dados, automação e integração com sistemas computacionais (Matthes, 2023). Além disso, serão utilizadas bibliotecas específicas para comunicação com os bancos de dados, permitindo execução de consultas e manipulação das informações extraídas.

Para conexão com o PostgreSQL será utilizada a biblioteca psycopg2, responsável por estabelecer comunicação entre aplicações Python e bancos PostgreSQL (Psycopg Development Team, 2025). Já para conexão com o MongoDB será utilizada a biblioteca pymongo, desenvolvida para interação entre aplicações Python e bancos documentais MongoDB (MongoDB Inc., 2025).

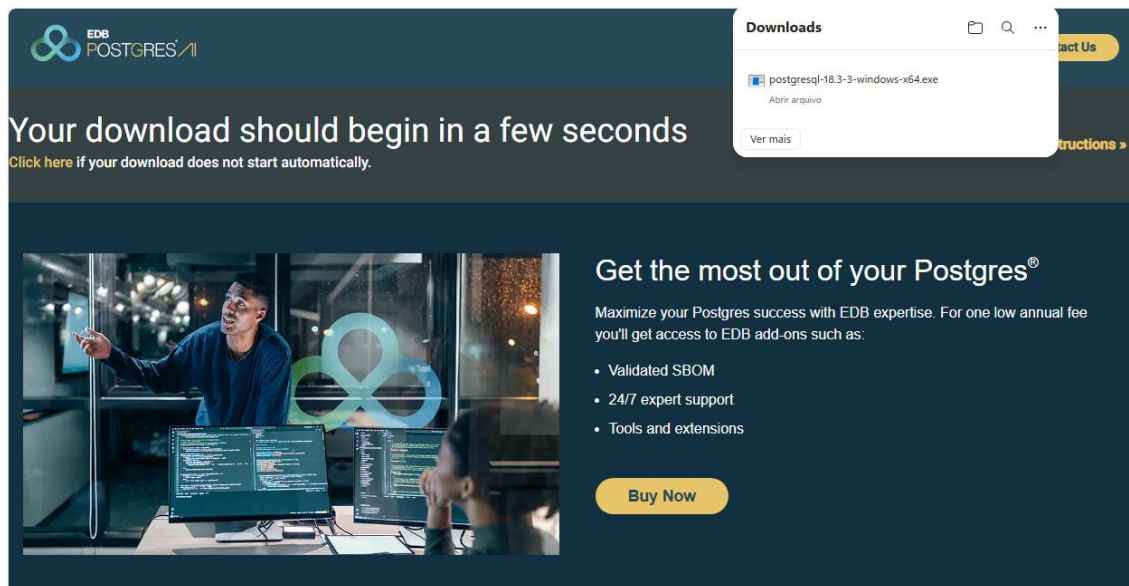
Além disso, será utilizada a biblioteca pandas para organização, tratamento e análise dos dados extraídos dos bancos de dados. Segundo McKinney (2022), o pandas é uma das principais bibliotecas utilizadas em ciência de dados e análise computacional devido à sua capacidade de manipulação eficiente de estruturas tabulares e datasets complexos.

As conexões gráficas com os bancos de dados serão realizadas utilizando as ferramentas pgAdmin e MongoDB Atlas/MongoDB Compass. O pgAdmin será utilizado para administração e visualização das tabelas relacionais do PostgreSQL, enquanto o MongoDB Atlas/MongoDB Compass será empregado para exploração das coleções documentais e análise estrutural do MongoDB (MongoDB Inc., 2025).

3.3 Procedimento operacional para conexão com PostgreSQL

O procedimento inicia-se com a instalação do PostgreSQL no computador. Para isso, o usuário deverá acessar o site oficial do PostgreSQL e realizar o download da versão compatível com o sistema operacional utilizado, como mostra na figura 1.

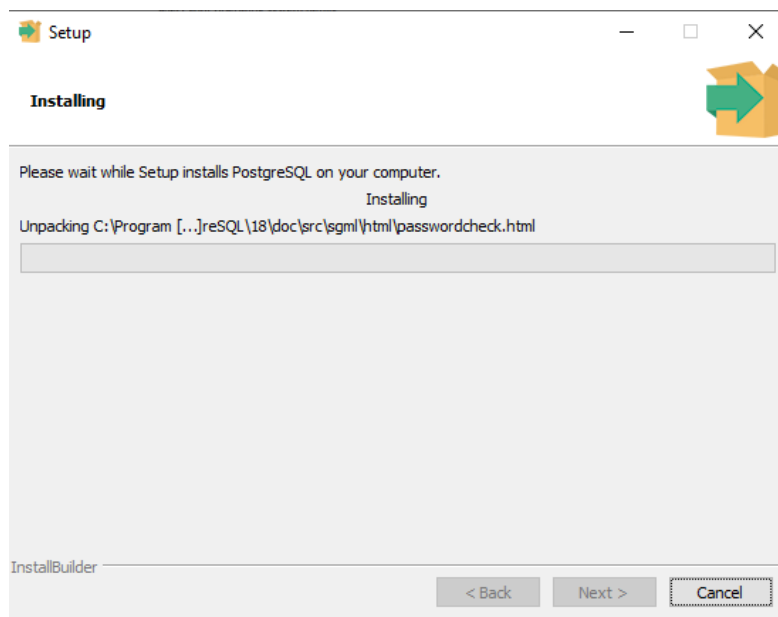
Figura 1 – Página para download do PostgreSQL



Fonte: PostgreSQL (2026).

Após concluir o download, deverá executar o instalador e seguir as instruções apresentadas na tela. Durante a instalação, recomenda-se manter os componentes padrão selecionados, incluindo PostgreSQL Server, pgAdmin e Command Line Tools.

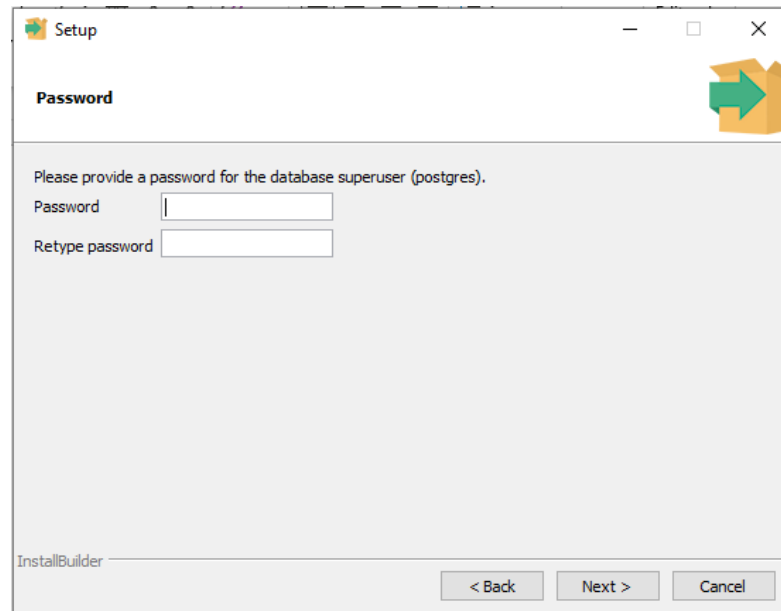
Figura 2 – Instalação do programa



Fonte: PostgreSQL (2026).

Em uma das etapas da instalação, o sistema solicitará a criação de uma senha para o usuário padrão denominado “postgres”.

Figura 3 - Criação de senha para usuário

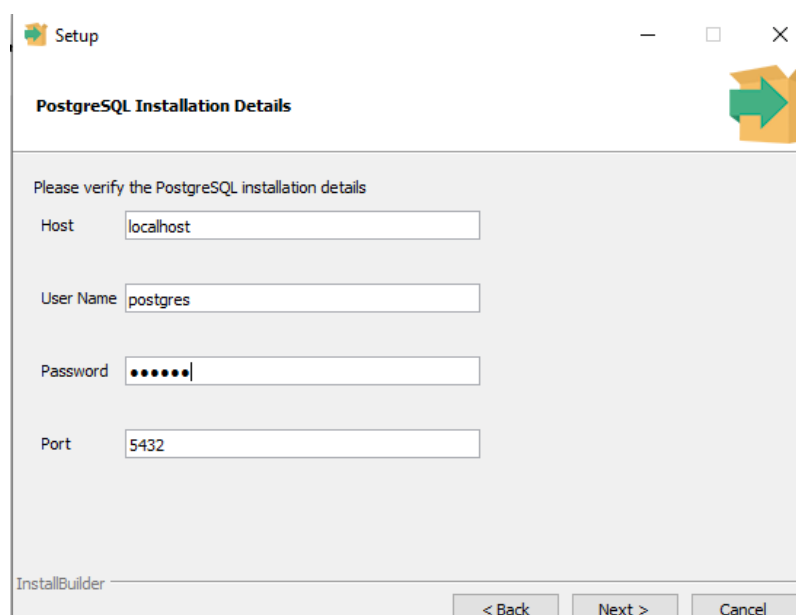


The screenshot shows a 'Setup' window titled 'Password'. It contains the text 'Please provide a password for the database superuser (postgres).' followed by two input fields: 'Password' and 'Retype password'. At the bottom, there are three buttons: '< Back', 'Next >', and 'Cancel'. The 'InstallBuilder' logo is visible in the bottom left corner.

Fonte: PostgreSQL (2026).

Essa senha deverá ser armazenada, pois será utilizada posteriormente durante as conexões com o banco de dados. Recomenda-se também manter a porta padrão “5432”, utilizada nativamente pelo PostgreSQL para comunicação entre aplicações e o servidor do banco de dados. Segundo Elmasri e Navathe (2018), a configuração adequada do ambiente é fundamental para garantir estabilidade e funcionamento correto do banco relacional.

Figura 4 – Detalhes para instalação

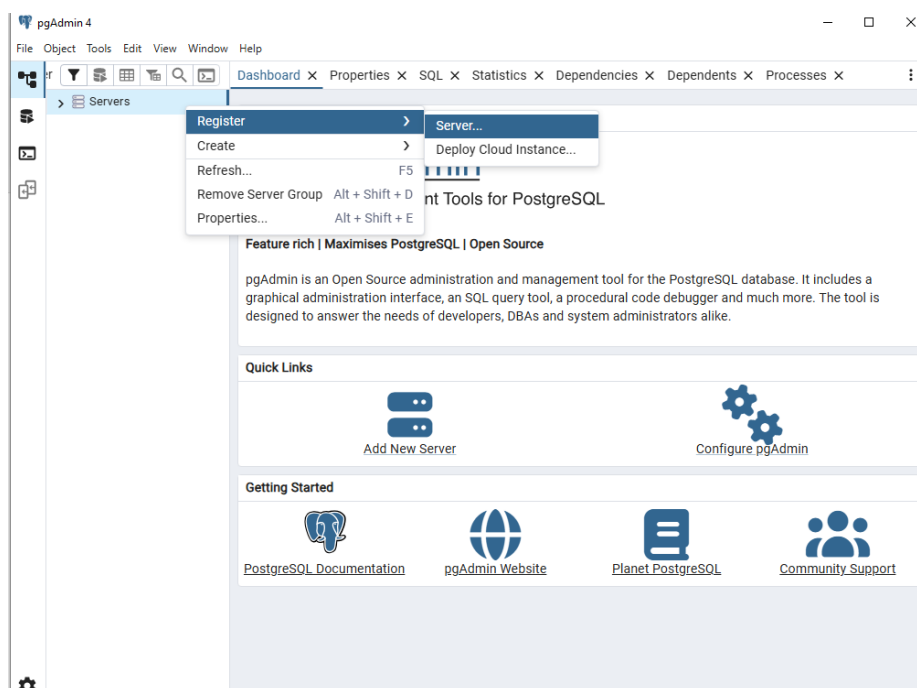


The screenshot shows a 'Setup' window titled 'PostgreSQL Installation Details'. It contains the text 'Please verify the PostgreSQL installation details' followed by four input fields: 'Host' (with 'localhost' pre-filled), 'User Name' (with 'postgres' pre-filled), 'Password' (with masked characters '•••••'), and 'Port' (with '5432' pre-filled). At the bottom, there are three buttons: '< Back', 'Next >', and 'Cancel'. The 'InstallBuilder' logo is visible in the bottom left corner.

Fonte: PostgreSQL (2026).

Após finalizar a instalação, o usuário deverá abrir o pgAdmin, ferramenta gráfica utilizada para administração do PostgreSQL. Ao abrir o sistema pela primeira vez, será necessário registrar uma conexão com o servidor local instalado anteriormente. Para isso, o usuário deverá localizar a seção “Servers”, clicar com o botão direito do mouse e selecionar a opção “Register” seguida de “Server”.

Figura 5 – Registrando conexão com servidor

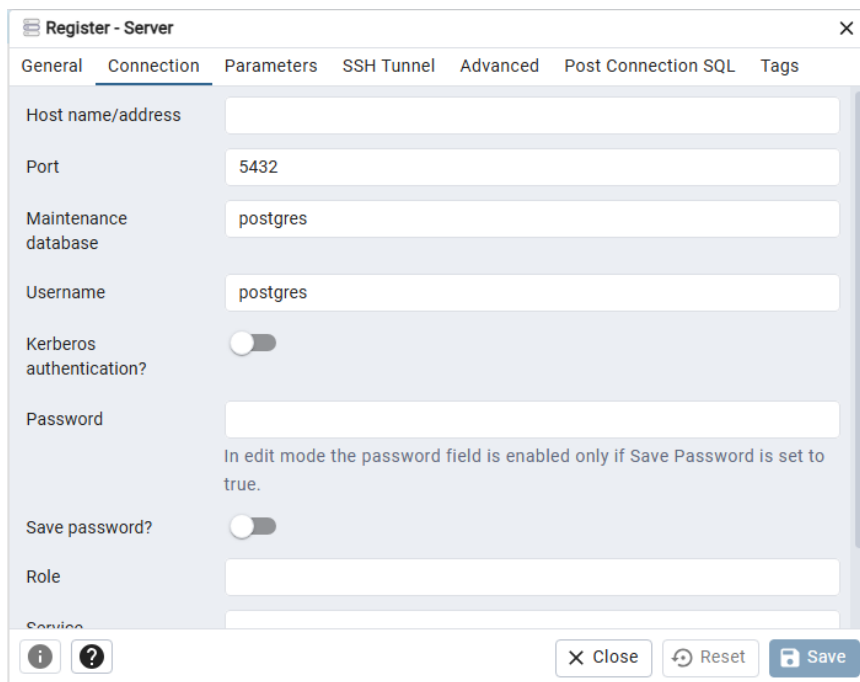


Fonte: PostgreSQL (2026).

Na janela de configuração da conexão, deverá ser definido um nome para identificação do servidor. Em seguida, na aba “Connection”, deverão ser preenchidos os seguintes parâmetros:

- Host: localhost ou endereço IP
- Port: 5432
- Maintenance database: Nome do Banco
- Username: postgres
- Password: senha definida durante a instalação

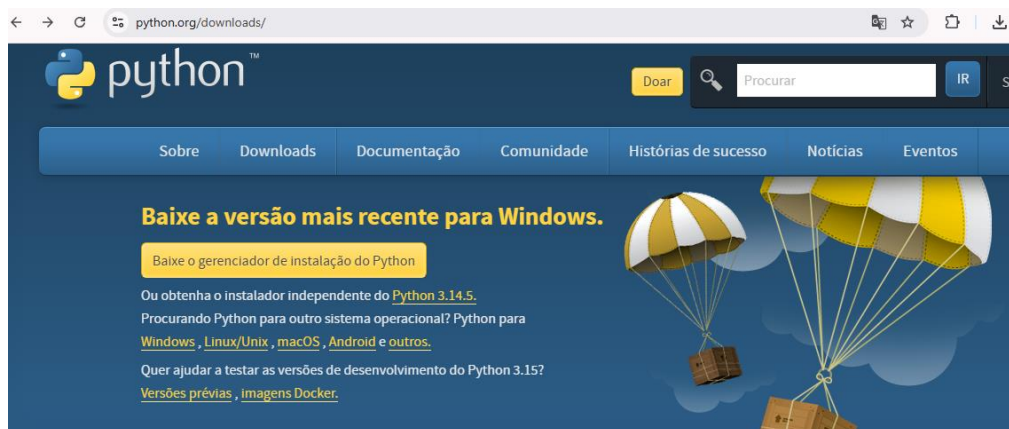
Figura 6 – Preenchimento de parâmetros



Fonte: PostgreSQL (2026).

Após preencher as informações, o usuário deverá clicar em “Save”. Caso a configuração tenha sido realizada corretamente, o servidor será exibido no painel lateral esquerdo do pgAdmin. Segundo PostgreSQL Global Development Group (2025), o pgAdmin permite executar comandos SQL, visualizar tabelas e administrar bancos relacionais de maneira visual e intuitiva. Concluída a configuração do PostgreSQL, será necessária a instalação da linguagem Python, utilizada para realização das conexões programáticas e análises dos dados. Para isso, o usuário deverá acessar o site oficial do Python, realizar o download da versão mais recente e executar o instalador.

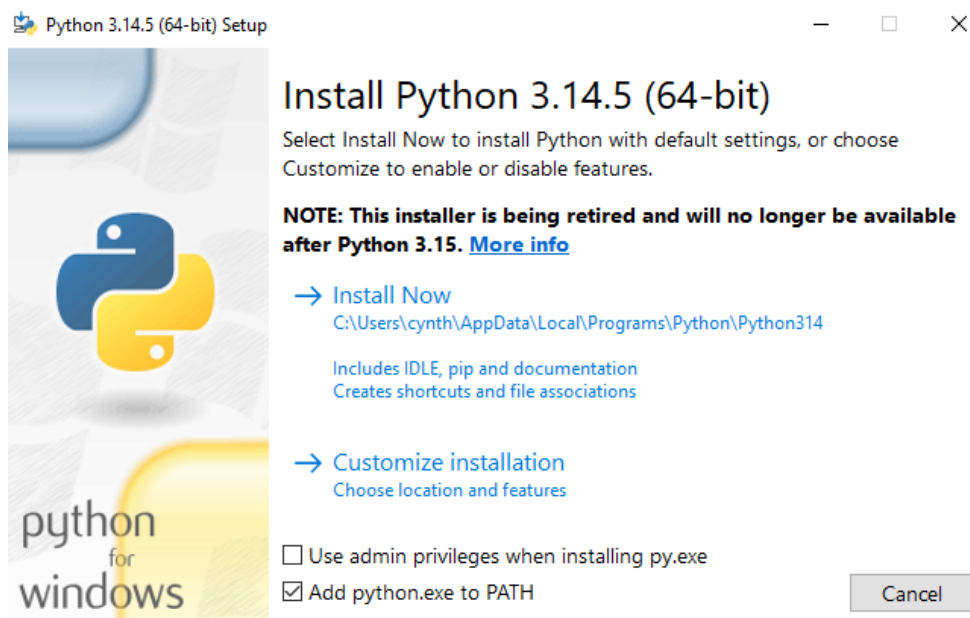
Figura 7 – Página para download do Python



Fonte: Python (2026).

Durante a instalação, recomenda-se marcar a opção “Add Python to PATH”, permitindo que o Python seja reconhecido automaticamente pelo sistema operacional.

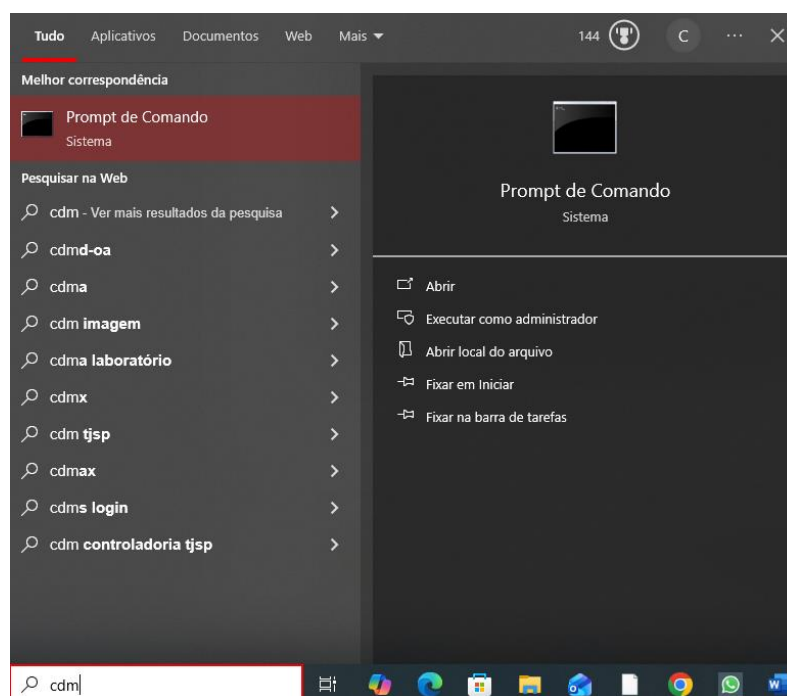
Figura 8 – Opção “Add Python to PATH” marcada no instalador



Fonte: Python (2026).

Após instalação do Python, o próximo passo consiste na instalação das bibliotecas responsáveis pela comunicação entre a aplicação e o banco de dados PostgreSQL. O usuário deverá abrir o Prompt de Comando ou Terminal do sistema operacional e executar os seguintes comandos:

Figura 9 – Abrindo o Prompt de Comando



Fonte: Elaborado pelos autores (2026).

Figura 10 – Instalando biblioteca psycpg2

```
cmd: Selecionar Prompt de Comando - pip install pandas

(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\cynth>pip install psycpg2
Collecting psycpg2
  Downloading psycpg2-2.9.12-cp314-cp314-win_amd64.whl.metadata (5.1 kB)
  Downloading psycpg2-2.9.12-cp314-cp314-win_amd64.whl (2.8 MB)
----- 2.8/2.8 MB 9.8 MB/s 0:00:00
Installing collected packages: psycpg2
Successfully installed psycpg2-2.9.12
```

Fonte: Elaborado pelos autores (2026).

Figura 11 – Instalando biblioteca pandas

```
cmd: Prompt de Comando

C:\Users\cynth>pip install pandas
Collecting pandas
  Downloading pandas-3.0.2-cp314-cp314-win_amd64.whl.metadata (19 kB)
Collecting numpy>=2.3.3 (from pandas)
  Downloading numpy-2.4.4-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
Collecting python-dateutil>=2.8.2 (from pandas)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting tzdata (from pandas)
  Downloading tzdata-2026.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
  Downloading pandas-3.0.2-cp314-cp314-win_amd64.whl (9.9 MB)
----- 9.9/9.9 MB 10.1 MB/s 0:00:01
  Downloading numpy-2.4.4-cp314-cp314-win_amd64.whl (12.4 MB)
----- 12.4/12.4 MB 10.8 MB/s 0:00:01
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
  Downloading tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Installing collected packages: tzdata, six, numpy, python-dateutil, pandas
Successfully installed numpy-2.4.4 pandas-3.0.2 python-dateutil-2.9.0.post0 six-1.17.0 tzdata-2026.2
```

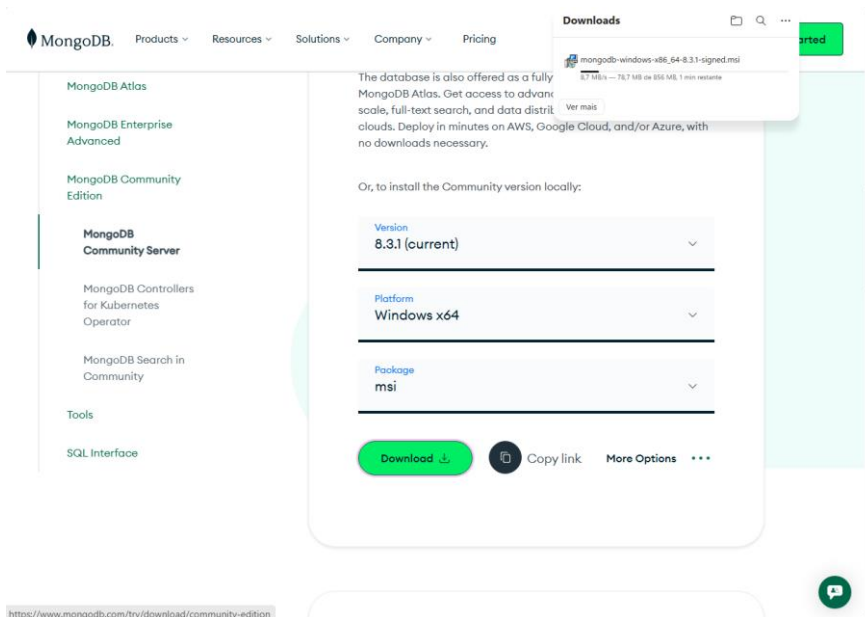
Fonte: Elaborado pelos autores (2026).

A biblioteca psycpg2 será responsável pela conexão entre Python e PostgreSQL, enquanto a biblioteca pandas será utilizada para organização e manipulação dos dados extraídos do banco de dados (Psycpg Development Team, 2025).

3.4 Procedimento operacional para conexão com MongoDB

O procedimento inicia-se com o download do MongoDB. Para isso, o usuário deverá acessar o site oficial do MongoDB e selecionar a versão compatível com o sistema operacional utilizado.

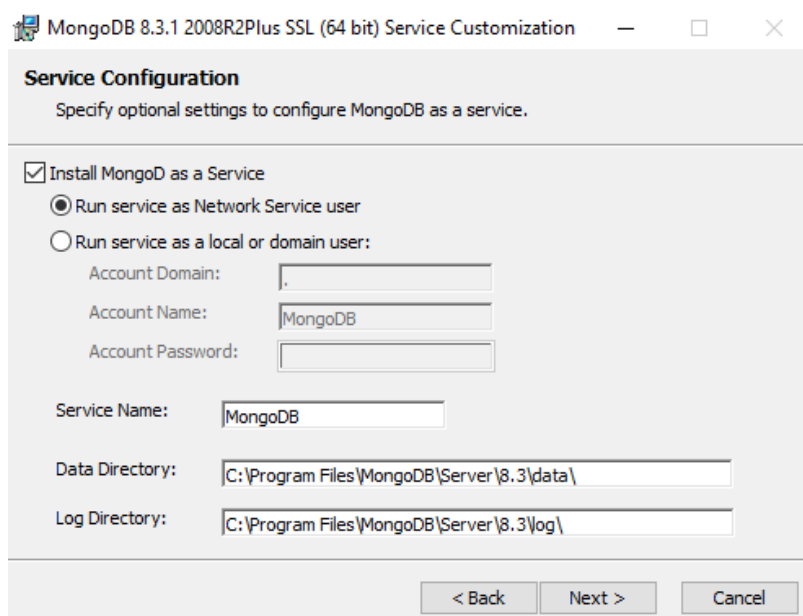
Figura 12 – Página para download do MongoDB



Fonte: MongoDB (2026).

Após concluir o download, deverá executar o instalador e seguir as etapas apresentadas na tela, mantendo as configurações padrão recomendadas pelo sistema.

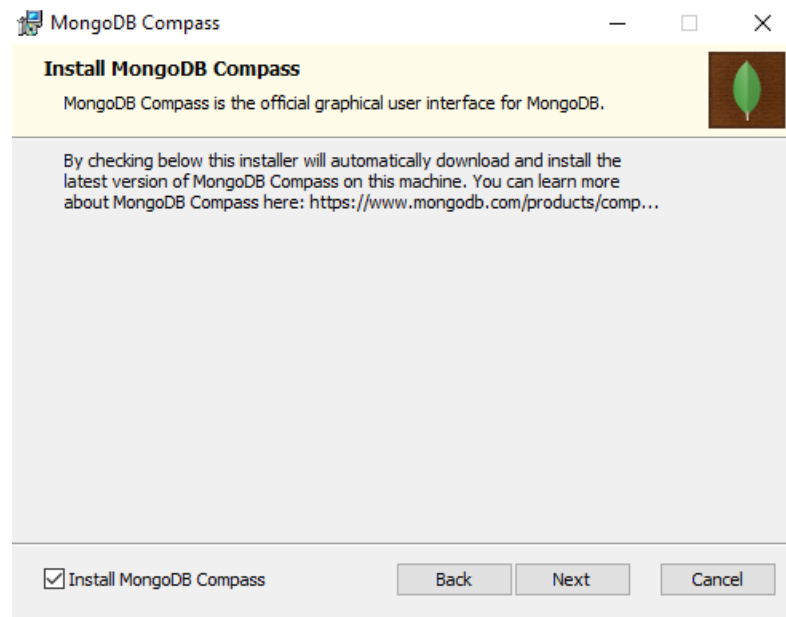
Figura 13 – Detalhes para instalação



Fonte: MongoDB (2026).

Durante a instalação, recomenda-se manter selecionadas as opções relacionadas ao MongoDB Server e ao MongoDB Compass.

Figura 14 – Opção “*Install MongoDB Compass*” marcada no instalador



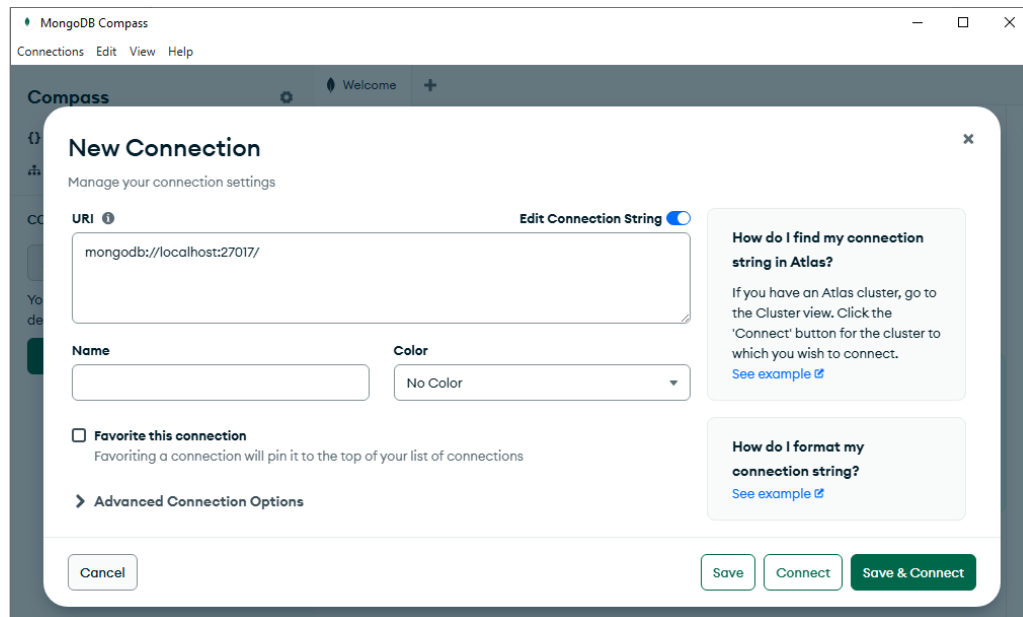
Fonte: MongoDB (2026).

O MongoDB Server será responsável pelo funcionamento do banco de dados, enquanto o MongoDB Compass será utilizado como ferramenta gráfica para visualização das coleções, documentos e bases de dados armazenadas no sistema. Segundo Banker (2011), ferramentas gráficas contribuem para facilitar a administração e exploração dos dados em ambientes não relacionais.

Após finalizar a instalação, o usuário deverá verificar se o serviço do MongoDB se encontra em execução no sistema operacional. Em muitos casos, o serviço é iniciado automaticamente após a instalação. Caso isso não ocorra, o usuário deverá acessar o gerenciador de serviços do sistema operacional e localizar o serviço denominado “MongoDB Server”. Em seguida, deverá clicar na opção “Iniciar”.

Com o servidor ativo, o próximo passo consiste em abrir o MongoDB Compass para realização da conexão gráfica com o banco de dados. Ao iniciar o programa, será exibido um botão denominado “*New Connection*”. A figura 15 mostra o botão mencionado.

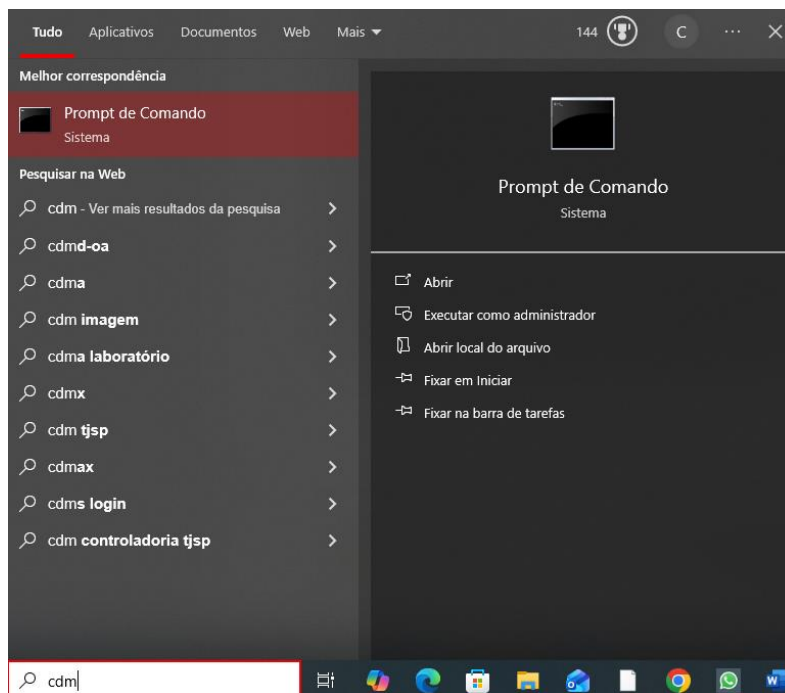
Figura 15 – Botão “*New Connection*”



Fonte: MongoDB (2026).

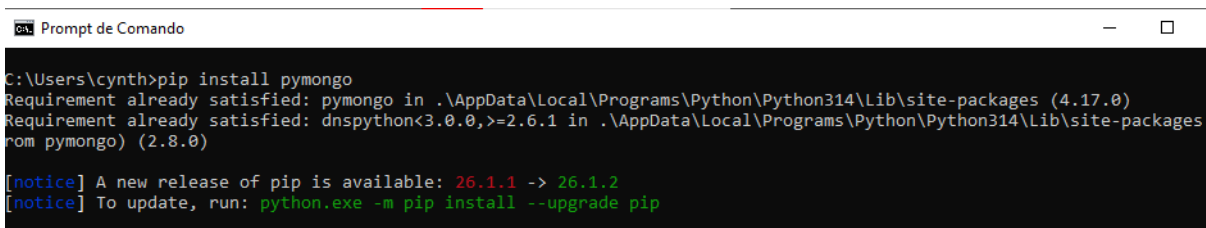
Após verificar o endereço, o usuário deverá clicar no botão “Connect”. Caso a conexão seja realizada corretamente, as bases de dados disponíveis serão exibidas no painel lateral esquerdo da aplicação. Segundo MongoDB Inc. (2025), a porta “27017” corresponde à porta padrão utilizada pelo MongoDB para comunicação entre aplicações e o servidor do banco de dados. Após estabelecimento da conexão gráfica, será necessária a instalação das bibliotecas utilizadas para comunicação entre Python e MongoDB. Para isso, o usuário deverá abrir o Prompt de Comando ou Terminal do sistema operacional e executar os seguintes comandos:

Figura 16 – Abrindo o Prompt de Comando



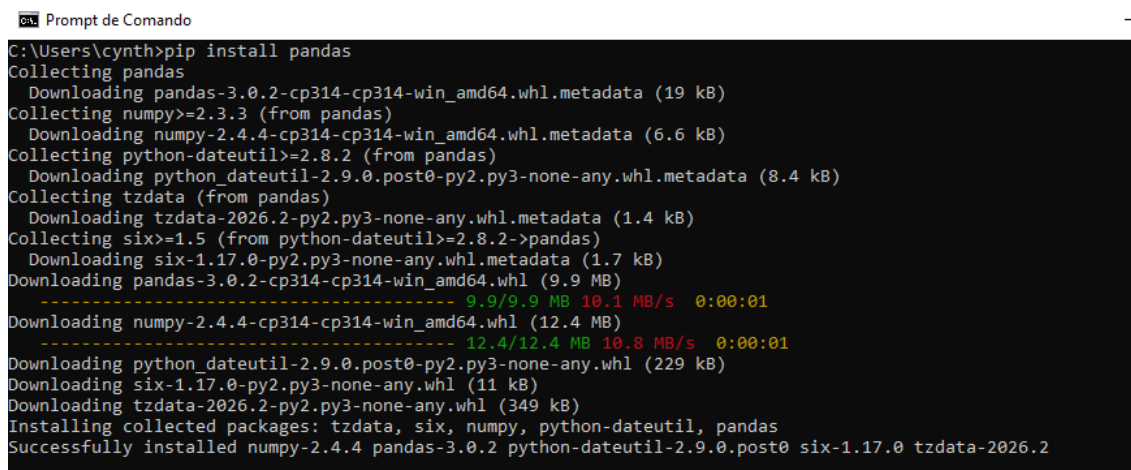
Fonte: Elaborado pelos autores (2026).

Figura 17 – Instalando biblioteca pymongo



Fonte: Elaborado pelos autores (2026).

Figura 18 – Instalando biblioteca pandas



Fonte: Elaborado pelos autores (2026).

A biblioteca pymongo será responsável pela integração entre Python e MongoDB, permitindo execução de consultas, recuperação de documentos e manipulação das coleções armazenadas no banco de dados. Já a biblioteca pandas será utilizada para organização e tratamento das informações extraídas (MongoDB Inc., 2025).

Após estabelecer conexão com o servidor, será possível acessar bases de dados e coleções específicas. Diferentemente dos bancos relacionais, o MongoDB trabalha com documentos organizados em coleções, não utilizando tabelas estruturadas com linhas e colunas (Chodorow, 2019). Essa característica proporciona maior flexibilidade no armazenamento dos dados.

4. Procedimento prático de conexão e extração

A primeira etapa deste trabalho (N1) consolidou a fundamentação teórica necessária à compreensão dos sistemas de informação, da arquitetura de software e dos modelos de banco de dados relacional e não relacional, além de documentar o procedimento de instalação e configuração dos ambientes PostgreSQL e MongoDB. A presente seção avança dessa etapa de configuração para a etapa de execução efetiva do Procedimento Operacional Padrão (POP), na qual os conceitos teóricos são aplicados sobre bases de dados reais, disponibilizadas em ambiente de nuvem para fins didáticos.

Diferentemente do ambiente local utilizado na N1 - no qual a instalação e a conexão foram realizadas no próprio computador do usuário - a presente etapa utiliza um ambiente remoto, hospedado em servidores próprios da disciplina, disponíveis de forma contínua (24 horas por dia, 7 dias por semana). Essa característica aproxima o procedimento da realidade de ambientes corporativos, nos quais os bancos de dados normalmente residem em servidores dedicados, acessados remotamente por diferentes aplicações e usuários (Begg, 2015). Foram utilizadas duas bases de dados distintas, cada uma representativa de um contexto de aplicação típico da Engenharia de Produção:

- "erp" - banco de dados relacional, hospedado em servidor PostgreSQL, representando um sistema de gestão empresarial (Enterprise Resource Planning), amplamente utilizado para controle de processos administrativos, financeiros, de estoque e produção (Laudon, 2014);
- "mes" - banco de dados não relacional, hospedado em servidor MongoDB, representando um sistema de execução da manufatura (*Manufacturing Execution*

System), tipicamente associado ao monitoramento de processos produtivos em tempo real, coleta de dados de chão de fábrica e rastreabilidade da produção (Helbig, 2013).

A escolha de duas bases com naturezas distintas - uma estruturada e transacional (ERP), outra voltada à captura de eventos operacionais (MES) - permite explorar, na prática, a discussão teórica apresentada na seção 2.4 sobre a complementaridade entre bancos relacionais e não relacionais em ambientes industriais, incluindo a estratégia de polyglot persistence descrita por Fowler e Sadalage (2012).

4.1 Conexão com o banco de dados relacional (PostgreSQL - "erp")

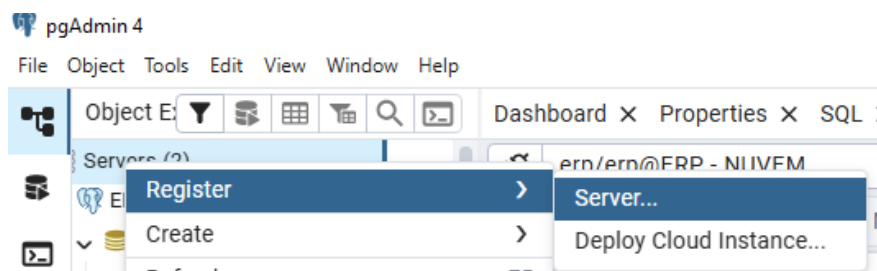
A conexão com o banco de dados relacional "erp" foi realizada por meio da ferramenta gráfica pgAdmin 4, ambiente de administração amplamente utilizado para gerenciamento de bancos de dados PostgreSQL. Diferentemente da etapa anterior, na qual foram realizadas atividades de instalação e configuração local do sistema gerenciador de banco de dados, nesta etapa foi utilizado um servidor remoto disponibilizado pela disciplina, permitindo a conexão a uma base de dados real hospedada em ambiente de nuvem.

A utilização do pgAdmin possibilitou a navegação visual pela estrutura do banco de dados, incluindo a identificação do servidor, do banco de dados, dos schemas, das tabelas, das colunas, dos tipos de dados e das restrições existentes. Essa abordagem reproduz práticas frequentemente adotadas em ambientes organizacionais, nos quais administradores e analistas de dados utilizam ferramentas gráficas para explorar, administrar e consultar bancos de dados corporativos (Begg, 2015).

O procedimento de conexão foi executado conforme as seguintes etapas:

1. Inicialmente, o pgAdmin 4 foi aberto e, na área de gerenciamento de servidores, selecionou-se a opção Register → Server para cadastrar uma nova conexão.

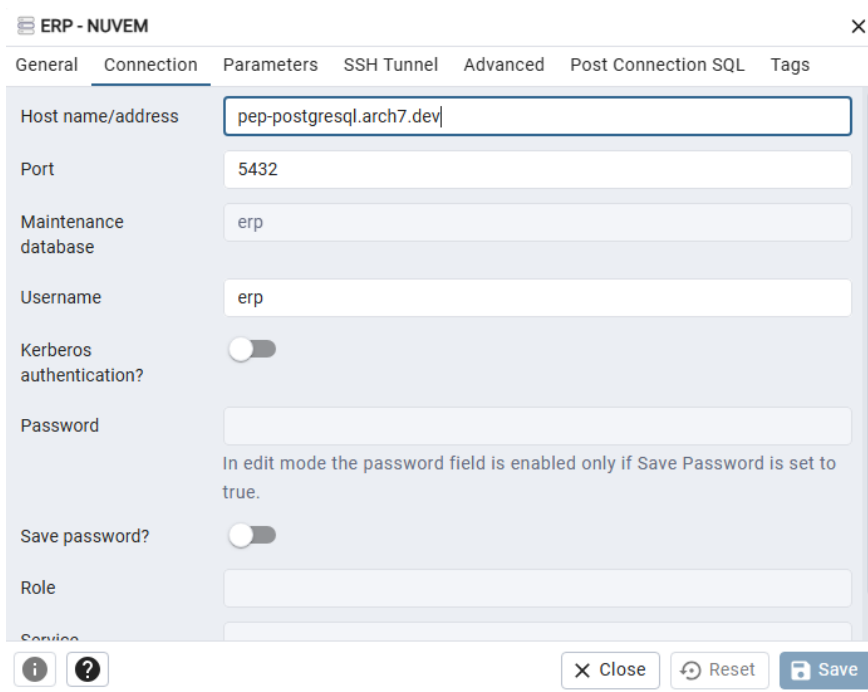
Figura 19 - Conectando ao Servidor no pgAdmin



Fonte: Elaborado pelos autores (2026).

2. Na aba General, foi atribuído um nome de identificação ao servidor, denominado ERP - NUVEM.
3. Na aba Connection, foram informados os parâmetros de acesso disponibilizados pela disciplina:
 - **Host name/address:** pep-postgresql.arch7.dev;
 - **Port:** 5432;
 - **Maintenance database:** erp;
 - **Username:** erp;
 - **Password:** Senha fornecida pelo professor.

Figura 20 - Configuração da conexão com o servidor PostgreSQL



Fonte: Elaborado pelos autores (2026).

4. Após o preenchimento das informações de conexão, acionou-se o comando Save, estabelecendo-se a comunicação com o servidor PostgreSQL remoto.
5. Com a conexão estabelecida, realizou-se a expansão hierárquica do servidor no painel de navegação do pgAdmin, seguindo a sequência: Servers → ERP → Databases → erp → Schemas → erp → Tables.

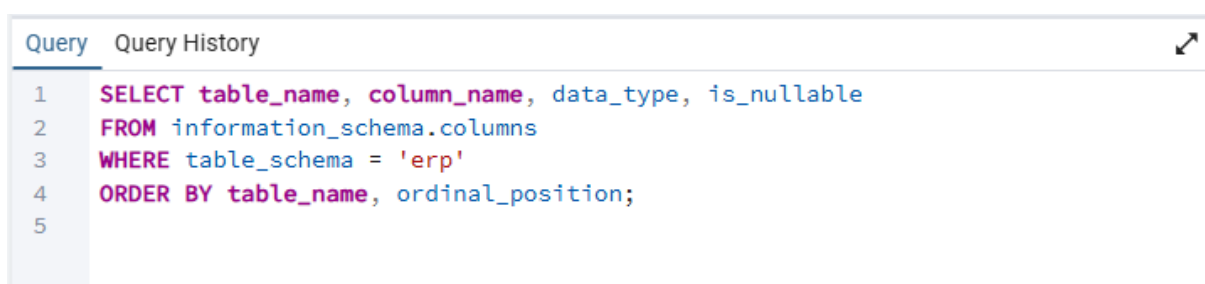
Esse procedimento permitiu visualizar as tabelas pertencentes ao schema "erp" e explorar individualmente suas características estruturais.

Entretanto, visando obter uma visão consolidada de toda a estrutura do banco de dados de maneira mais eficiente, utilizou-se o ambiente de execução de consultas SQL (Query Tool) do pgAdmin. Para acessar essa funcionalidade, selecionou-se o banco de dados "erp" no painel de

navegação e, por meio do menu de contexto (botão direito do mouse), acionou-se a opção Query Tool.

Nesse ambiente, foi executada uma consulta ao catálogo de metadados do PostgreSQL (*information_schema*), responsável por armazenar informações descritivas sobre os objetos do banco de dados. A consulta executada permitiu identificar simultaneamente todas as tabelas pertencentes ao schema "erp", bem como seus respectivos atributos, tipos de dados e restrições de nulidade, conforme apresentado na figura 21.

Figura 21 - Código para consulta SQL para mapeamento estrutural do schema "erp"



```
Query  Query History
1  SELECT table_name, column_name, data_type, is_nullable
2  FROM information_schema.columns
3  WHERE table_schema = 'erp'
4  ORDER BY table_name, ordinal_position;
5
```

Fonte: Elaborado pelos autores (2026).

Após a inserção da consulta no Query Tool, o comando foi executado por meio da tecla F5 ou do botão de execução (*Execute/Play*) disponível na barra de ferramentas do pgAdmin. O resultado foi apresentado na região inferior da interface, em formato tabular, permitindo a análise simultânea de toda a estrutura do schema.

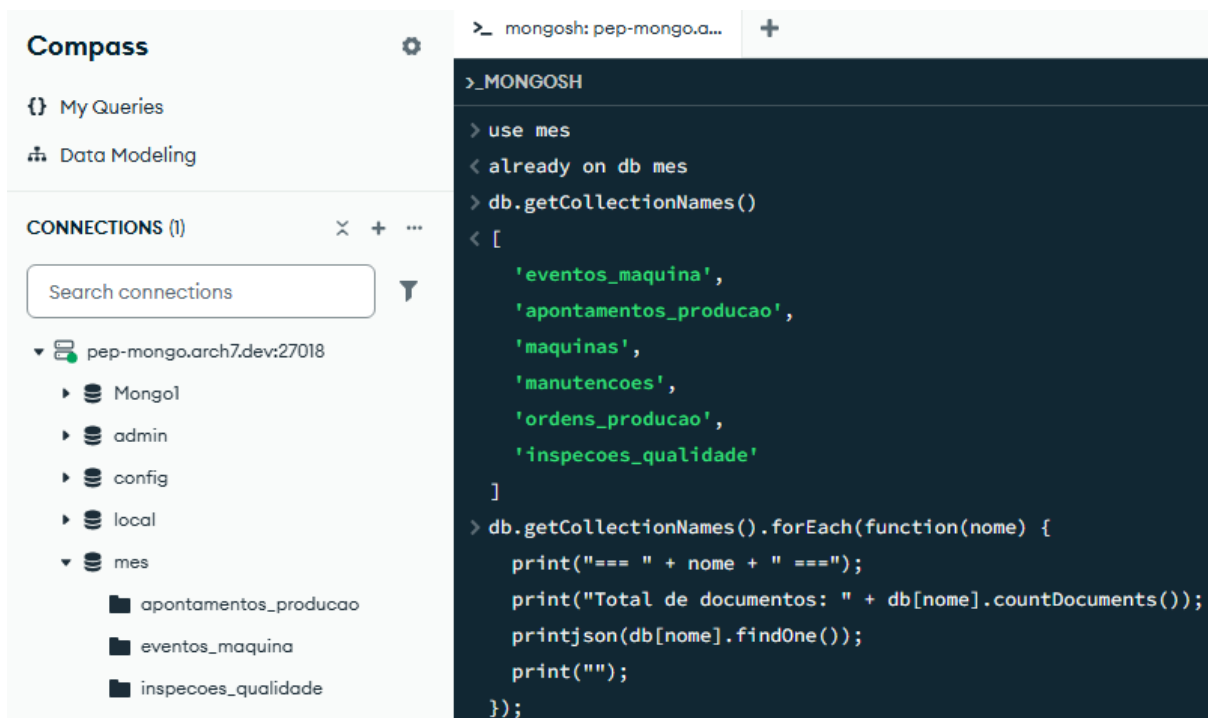
As informações obtidas possibilitaram a identificação das entidades do sistema, de seus atributos, dos tipos de dados associados e das restrições existentes. Esses elementos constituíram a base para a construção do diagrama entidade-relacionamento que será apresentado na Seção 5, bem como para a identificação das chaves primárias e estrangeiras e para a posterior análise das formas normais aplicadas à base de dados relacional.

4.2 Conexão com o banco de dados não relacional (MongoDB - "mes")

A conexão com o banco "mes" foi estabelecida utilizando o MongoDB Shell (mongosh), integrado à ferramenta MongoDB Compass, por meio da string de conexão autenticada no próprio banco "mes" (authSource=admin). O mapeamento das coleções existentes foi realizado por meio do comando `db.getCollectionNames()`, complementado pela inspeção de um

documento de exemplo de cada coleção (findOne()) e da contagem total de documentos (countDocuments()), conforme apresentado na Figura 22.

Figura 22 - Coleções do banco de dados “mes”



Fonte: Elaborado pelos autores (2026).

A execução do procedimento revelou seis coleções, cujo volume de documentos e finalidade estão descritos na Tabela 1.

Tabela 1 - Quantidade de documentos na coleções do banco de dados “mes”

Coleção	Documentos	Finalidade
eventos_maquina	6.373	Registro de eventos de chão de fábrica (estado da máquina por turno)
apontamentos_producao	2.088	Apontamento de produção por ordem, com paradas detalhadas
maquinas	4	Cadastro das máquinas/recursos produtivos
manutencoes	207	Registro de manutenções preventivas e corretivas
ordens_producao	96	Ordens de produção (espelha a tabela "ordem_producao" do erp)

Coleção	Documentos	Finalidade
inspecoes_qualidade	192	Inspeções de qualidade por amostragem

Fonte: Elaborado pelos autores (2026).

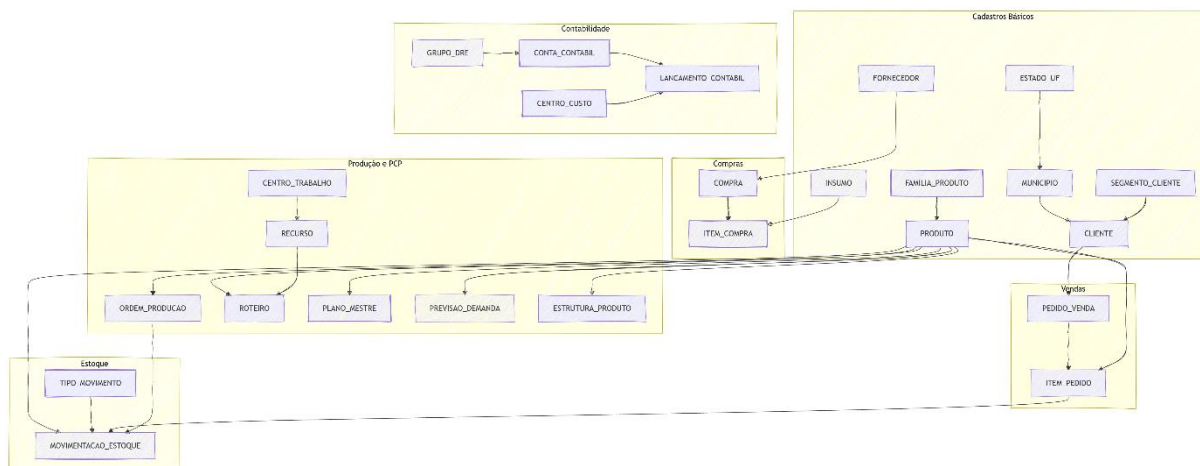
Observa-se que a coleção "ordens_producao" apresenta o mesmo volume de registros (96) identificado na tabela "ordem_producao" do banco relacional "erp", o que evidencia a integração entre os dois sistemas: o ERP mantém o registro administrativo e financeiro da ordem de produção, enquanto o MES mantém o registro operacional detalhado de sua execução em chão de fábrica, ilustrando, na prática, a estratégia de polyglot persistence discutida por Fowler e Sadalage (2012), na qual diferentes bancos de dados coexistem na mesma organização, cada um atendendo a uma necessidade específica.

5. Modelagem e normalização aplicadas à base de dados real

A construção do diagrama entidade-relacionamento e a verificação das formas normais seguiram o procedimento descrito por Elmasri e Navathe (2018). Inicialmente, foram identificadas as tabelas pertencentes ao schema "erp" por meio da navegação no ambiente pgAdmin 4 e pela execução da consulta de mapeamento estrutural apresentada na Figura 21. Em seguida, realizou-se a inspeção das informações retornadas pelo catálogo de metadados do PostgreSQL, identificando-se os atributos, os tipos de dados e as restrições de cada tabela. Com base nessas informações, procedeu-se à identificação das chaves primárias e estrangeiras e ao mapeamento dos relacionamentos existentes entre as entidades, permitindo determinar as cardinalidades presentes no modelo de dados. Posteriormente, cada tabela foi analisada quanto à conformidade com a Primeira Forma Normal (1FN), Segunda Forma Normal (2FN) e Terceira Forma Normal (3FN), verificando-se, respectivamente, a atomicidade dos atributos, a inexistência de dependências parciais em relação às chaves candidatas e a ausência de dependências transitivas entre atributos não chave.

A partir da conexão feita foi possível mapear a estrutura completa do schema "erp", composto por 26 tabelas relacionadas entre si por chaves primárias e estrangeiras. A Figura 23 apresenta o diagrama entidade-relacionamento (ER) construído a partir desse mapeamento, evidenciando as principais entidades do banco de dados e os relacionamentos existentes entre elas.

Figura 23 - Diagrama entidade-relacionamento do banco de dados "erp"



Fonte: Elaborado pelos autores (2026).

5.1 Aplicação das formas normais

A análise da estrutura mapeada demonstrou que o schema "erp" encontra-se em conformidade com a Primeira Forma Normal (1FN), uma vez que todos os atributos das tabelas analisadas armazenam valores atômicos, não havendo campos com listas, grupos repetitivos ou múltiplos valores em uma mesma célula. Não foram identificadas violações da 1FN em nenhuma das 26 tabelas do schema.

Em relação à Segunda Forma Normal (2FN), destacam-se as tabelas "estrutura_produto", "plano_mestre" e "previsao_demanda", por utilizarem chaves primárias compostas. A tabela "estrutura_produto", por exemplo, possui chave primária composta pelos atributos "produto_pai_id" e "componente_id", representando a estrutura de produto (*Bill of Materials*) de um item. O atributo não chave "quantidade" depende integralmente da combinação dos dois atributos da chave composta -ou seja, depende de qual produto pai está associado a qual componente, não havendo dependência parcial em relação a apenas um dos atributos da chave. O mesmo padrão se repete nas tabelas "plano_mestre" e "previsao_demanda", cuja chave composta é formada por "produto_id" e "ano_mes", sendo os atributos "qtd_planejada" e "qtd_prevista", respectivamente, dependentes da combinação completa da chave. Dessa forma, o schema "erp" está em conformidade com a 2FN.

Quanto à Terceira Forma Normal (3FN), observa-se que o modelo evita sistematicamente dependências transitivas por meio da substituição de atributos descritivos por chaves estrangeiras. Um exemplo relevante ocorre na tabela "cliente", que armazena os atributos "segmento_id" e "municipio_id" como referências às tabelas "segmento_cliente" e "municipio",

em vez de armazenar diretamente o nome do segmento ou o nome do município na própria tabela "cliente". Caso o nome do município estivesse duplicado na tabela "cliente", uma atualização no nome do município exigiria alteração em todos os registros de clientes associados a ele, configurando dependência transitiva e violação da 3FN - o que não ocorre no modelo analisado. Padrão equivalente é observado na tabela "conta_contabil", que referência "grupo_dre_codigo" em vez de duplicar o nome do grupo de Demonstração de Resultado do Exercício (DRE) em cada conta contábil. Conclui-se, portanto, que o schema "erp" está em conformidade com a 3FN.

5.2 Relacionamentos identificados

O mapeamento da base "erp" permitiu identificar exemplos concretos dos três tipos de relacionamento descritos na fundamentação teórica (seção 2.5):

- Relacionamento 1:N: ocorre, por exemplo, entre "familia_produto" e "produto" -uma família de produtos agrupa diversos produtos, enquanto cada produto pertence a uma única família (atributo "familia_id" em "produto"). Relacionamento semelhante ocorre entre "cliente" e "pedido_venda": um cliente pode realizar múltiplos pedidos, mas cada pedido está vinculado a um único cliente.
- Relacionamento N:M: identificado na relação entre "pedido_venda" e "produto", resolvida pela tabela associativa "item_pedido", que armazena os atributos "quantidade", "preco_unitario" e "desconto_pct" referentes à combinação específica de um pedido com um produto. Relacionamento N:M equivalente ocorre entre "compra" e "insumo", resolvido pela tabela "item_compra".
- Relacionamento N:M recursivo (autorrelacionamento): a tabela "estrutura_produto" representa uma relação N:M do próprio "produto" consigo mesmo, modelando a estrutura de produto (Bill of Materials, ou árvore de componentes) típica de sistemas de Planejamento e Controle da Produção (PCP): um produto pai pode ser composto por diversos componentes, e um mesmo componente pode integrar a estrutura de diversos produtos pais.

5.3 Agregações e composições identificadas

Na base "erp", identificou-se uma relação de agregação entre "fornecedor" e "compra": embora uma compra esteja associada a um fornecedor específico ("fornecedor_id"), o fornecedor possui existência independente no banco de dados e continua existindo mesmo que todas as compras associadas a ele sejam removidas. O mesmo padrão de agregação ocorre entre "insumo" e "fornecedor" por meio do atributo opcional "fornecedor_padrao_id" - a nulidade desse campo (marcada como "YES" no mapeamento) reforça o caráter de associação independente, já que um insumo pode existir sem possuir fornecedor padrão definido.

Por outro lado, identificou-se uma relação de composição entre "pedido_venda" e "item_pedido": os registros de "item_pedido" não possuem sentido nem existência fora do contexto de um pedido específico, sendo diretamente dependentes dele. Caso um pedido seja removido do sistema, seus itens deveriam ser removidos em cascata, por meio da cláusula ON DELETE CASCADE discutida na seção 2.6. Relação de composição equivalente ocorre entre "compra" e "item_compra".

Um aspecto particular da modelagem foi observado na tabela "movimentacao_estoque", que possui dois atributos estrangeiros opcionais - "item_pedido_id" e "ordem_producao_id" - utilizados para registrar a origem da movimentação: uma saída de estoque pode ser originada por um item de pedido de venda, enquanto uma entrada pode ser originada por uma ordem de produção concluída. Essa estrutura evidencia, na prática, a necessidade de associar um mesmo tipo de evento (movimentação) a diferentes contextos de origem, relacionando-se diretamente à tabela "tipo_movimento", que classifica a natureza de cada movimentação.

5.4 Modelagem documental do banco "mes"

Diferentemente do banco "erp", estruturado em tabelas normalizadas, o banco "mes" organiza as informações em documentos, e a relação entre as coleções ocorre majoritariamente por meio de referência por chave de negócio (campos como "maquina_codigo", "produto_sku" e "ordem"), e não por ObjectId, como descrito teoricamente na seção 2.6. Essa escolha de modelagem é coerente com a necessidade de rastreabilidade entre sistemas: os campos "ordem" e "produto_sku" correspondem diretamente aos identificadores de negócio utilizados no banco relacional "erp" (respectivamente, "ordem_producao.codigo" e "produto.sku"), permitindo a correlação entre os dois bancos de dados sem depender de identificadores internos do MongoDB.

A coleção "apontamentos_producao" apresenta um exemplo claro de composição por meio de documentos embutidos (embedded documents): o campo "paradas" armazena um array de subdocumentos, cada um contendo "motivo" e "duracao_min". Essas paradas não possuem existência nem sentido fora do apontamento de produção ao qual pertencem -caso o apontamento seja removido, as paradas embutidas são removidas automaticamente junto com o documento principal, configurando uma relação de composição ("parte de"), conforme a distinção apresentada por Booch, Rumbaugh e Jacobson (2005). Esse modelo de documentos embutidos é vantajoso neste caso porque os apontamentos são consultados sempre em conjunto com suas paradas, evitando a necessidade de operações de JOIN -operação nativamente ausente no MongoDB -para reconstruir o evento de produção completo (Banker, 2011).

Já a relação entre "eventos_maquina" e "maquinas", assim como entre "manutencoes" e "maquinas", é implementada por agregação: o documento de evento ou de manutenção referencia a máquina por meio do campo "maquina_codigo", mas a máquina mantém existência própria e independente na coleção "maquinas", podendo ser consultada isoladamente e continuando a existir mesmo que todos os seus eventos ou manutenções sejam removidos.

A coleção "apontamentos_producao" também ilustra a prática de desnormalização proposital comum em bancos documentais: o documento armazena tanto o código do recurso ("recurso_codigo") quanto seu nome ("recurso_nome"), assim como tanto o SKU do produto ("produto_sku") quanto seu nome ("produto_nome"). Essa duplicação de dados, que seria considerada uma violação da Terceira Forma Normal em um banco relacional (por configurar dependência transitiva), é aceitável e até desejável no modelo documental, pois prioriza o desempenho de leitura -o documento pode ser consultado isoladamente, sem necessidade de operações de junção com outras coleções —, em detrimento da consistência forte e da ausência de redundância, características do modelo BASE discutido por Brewer (2012) em contraposição ao modelo ACID dos bancos relacionais.

5.5 Relacionamentos entre os bancos "erp" e "mes"

A análise conjunta das duas bases evidencia um fluxo de informação que se inicia no planejamento (banco "erp": "plano_mestre" e "ordem_producao") e se desdobra na execução (banco "mes": "ordens_producao", "apontamentos_producao" e "eventos_maquina"), sendo posteriormente realimentado pelo controle de qualidade ("inspecoes_qualidade") e pela

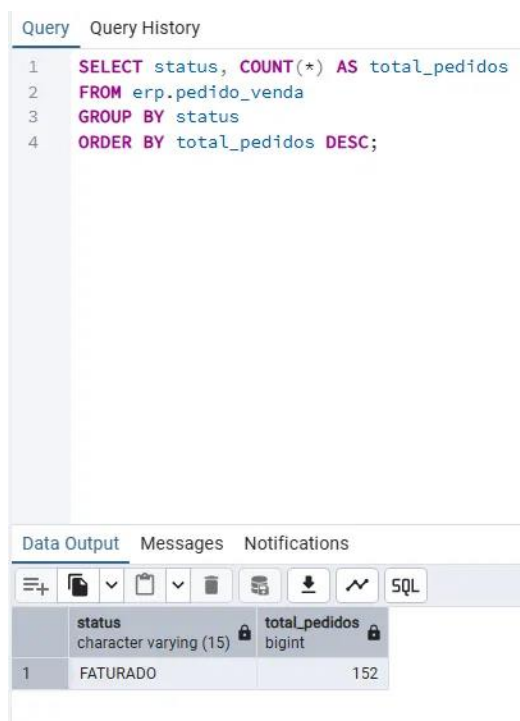
manutenção ("manutencoes"). Esse fluxo é coerente com a integração entre sistemas ERP e MES discutida por Kagermann, Wahlster e Helbig (2013) no contexto da Indústria 4.0, na qual o MES atua como camada intermediária entre o planejamento corporativo (ERP) e o controle físico do processo produtivo (chão de fábrica).

6. Consultas e indicadores extraídos do banco "erp"

A partir da conexão programática estabelecida na seção 4.1 e do mapeamento estrutural apresentado na seção 5, foram executadas consultas SQL direcionadas à extração de indicadores relevantes aos processos de vendas, compras, produção e controle financeiro -áreas centrais à atuação do engenheiro de produção (Slack; Brandon-Jones; Johnston, 2018). As consultas foram executadas por meio da ferramenta pgAdmin permitindo a organização tabular dos dados extraídos.

6.1 Indicador 1 -Distribuição de pedidos de venda por status

Figura 24 - Consulta de pedidos de venda por status



The screenshot shows the pgAdmin interface. The top pane displays a SQL query: `SELECT status, COUNT(*) AS total_pedidos FROM erp.pedido_venda GROUP BY status ORDER BY total_pedidos DESC;`. The bottom pane shows the 'Data Output' tab with a single result row.

	status character varying (15)	total_pedidos bigint
1	FATURADO	152

Fonte: Elaborado pelos autores (2026).

A consulta retornou um único status "FATURADO", totalizando 152 pedidos de venda. A ausência de outros status (como pedidos em aberto, em separação ou cancelados) indica que a

base de dados disponibilizada representa um recorte consolidado de pedidos já concluídos, adequado à análise histórica de faturamento e demanda, mas não permite, isoladamente, avaliar indicadores de eficiência operacional do ciclo de vendas, como taxa de cancelamento ou tempo médio entre a abertura e o faturamento do pedido. Tal limitação é discutida na seção de considerações finais deste trabalho.

6.2 Indicador 2 - Margem de estoque em relação ao estoque de segurança

Figura 25 - Produtos com risco de ruptura de estoque

Query	Query History
1	SELECT sku, nome, estoque_inicial, estoque_seguranca,
2	(estoque_inicial - estoque_seguranca) AS margem
3	FROM erp.produto
4	WHERE estoque_inicial < estoque_seguranca
5	ORDER BY margem ASC ;

Data Output	Messages	Notifications
+		
SQL		
sku character varying (20)	nome character varying (80)	estoque_inicial integer
		estoque_seguranca integer
		margem integer

Fonte: Elaborado pelos autores (2026).

A consulta que compara o estoque inicial de cada produto ao seu respectivo estoque de segurança não retornou nenhum registro, indicando que, na base de dados analisada, nenhum produto encontra-se com nível de estoque abaixo do limite de segurança estabelecido. Esse resultado sugere um cenário de gestão de estoque adequado, em conformidade com os princípios de planejamento e controle de estoques discutidos por Slack, Brandon-Jones e Johnston (2018), nos quais o estoque de segurança tem justamente a função de absorver variações de demanda sem gerar ruptura.

A fim de aprofundar essa análise, verificou-se a margem (diferença entre estoque inicial e estoque de segurança) dos dez produtos com menor folga, apresentados na Figura 25. Observa-se que os produtos com menor margem absoluta pertencem predominantemente à família de bombas e conjuntos pressurizadores (CJM-700, CJM-500, BMB-300, BMB-200 e BMB-100),

com margens entre 6 e 60 unidades, enquanto os itens da família de válvulas (VLV-100, VLV-075 e VLV-050) apresentam margens consideravelmente maiores, entre 250 e 500 unidades.

Figura 26 - Produtos com menor margem entre estoque inicial e estoque de segurança

Query

Query History

1

SELECT sku, nome, estoque_inicial, estoque_seguranca,
(estoque_inicial - estoque_seguranca) **AS** margem
FROM erp.produto
ORDER BY margem **ASC**
LIMIT 10;

Data Output

Messages

Notifications

≡+















SQL

Showing rows: 1 to 8





Page No:

	sku character varying (20)	nome character varying (80)	estoque_inicial integer	estoque_seguranca integer	margem integer
1	CJM-700	Conjunto Booster Duplo	12	6	6
2	CJM-500	Conjunto Pressurizador	20	10	10
3	BMB-300	Bomba Submersa 3cv	50	25	25
4	BMB-200	Bomba Centrífuga 2cv	80	40	40
5	BMB-100	Bomba Centrífuga 1cv	120	60	60
6	VLV-100	Válvula Gaveta 1"	450	200	250
7	VLV-075	Válvula Esfera 3/4"	700	300	400
8	VLV-050	Válvula Esfera 1/2"	900	400	500

Fonte: Elaborado pelos autores (2026).

Mais relevante do que a margem absoluta, contudo, é a proporção entre estoque inicial e estoque de segurança: em praticamente todos os itens listados, o estoque de segurança corresponde exatamente à metade do estoque inicial (por exemplo, a Bomba Centrífuga 1cv possui estoque inicial de 120 unidades e estoque de segurança de 60). Esse padrão sugere que o estoque de segurança foi definido como uma proporção fixa (50%) do estoque inicial, e não a partir de uma análise específica de variabilidade de demanda, lead time de reposição ou criticidade de cada item, conforme recomendado pela literatura de gestão de estoques (Slack; Brandon-Jones; Johnston, 2018). Embora essa abordagem simplifique o planejamento, ela pode resultar em níveis de segurança subdimensionados para itens de alta variabilidade de demanda e sobredimensionados para itens de demanda estável, representando uma oportunidade de refinamento da política de estoques da organização.

6.3 Indicador 3 - Faturamento total por cliente

Figura 27 - Consulta de faturamento total por cliente

Query

Query History

```
1  SELECT c.nome AS cliente,
2         SUM(ip.quantidade * ip.preco_unitario * (1 - ip.desconto_pct/100.0)) AS f
3  FROM   erp.pedido_venda pv
4  JOIN   erp.cliente c ON c.id = pv.cliente_id
5  JOIN   erp.item_pedido ip ON ip.pedido_id = pv.id
6  GROUP BY c.nome
7  ORDER BY faturamento_total DESC
8  LIMIT 10;
```

Data Output

Messages

Notifications

</

Fonte: Elaborado pelos autores (2026).

A Figura 27 apresenta os dez clientes com maior faturamento acumulado. Observa-se forte concentração de receita nos primeiros colocados: o cliente "Condomínio Parque das Águas" sozinho responde por R\$ 125.208.625,10, valor quase o dobro do segundo colocado ("Indústria Metalúrgica Bandeirantes", com R\$ 63.531.648,70) e mais de quatro vezes o décimo colocado ("Cervejaria Pinheiro Alto", com R\$ 26.351.411,60). Esse padrão de concentração é compatível com o princípio de Pareto (regra 80/20), frequentemente observado na análise de carteiras de clientes industriais, e sugere que a organização representada pela base "erp" possui alta dependência de poucos clientes-chave para a geração de receita. Do ponto de vista da gestão de risco comercial, essa concentração reforça a importância de estratégias de retenção e diversificação de carteira, especialmente para os clientes do topo do ranking.

6.4 Indicador 4 - Custo total de compras por fornecedor

Figura 28 - Consulta de custo total de compras por fornecedor

Query

Query History

```
1 SELECT f.nome AS fornecedor,
2       SUM(ic.quantidade * ic.custo_unitario) AS custo_total
3 FROM   erp.compra co
4 JOIN   erp.fornecedor f ON f.id = co.fornecedor_id
5 JOIN   erp.item_compra ic ON ic.compra_id = co.id
6 GROUP BY f.nome
7 ORDER BY custo_total DESC;
```

Data Output

Messages

Notifications

Showing rows: 1 to 6

	fornecedor character varying (80)	custo_total numeric
1	Motores Elétricos WEG-li...	1685990.89
2	Rolamentos & Cia	1582799.80
3	Vedações Selomax	1398832.60
4	Aços Especiais do Brasil	1285756.31
5	Polímeros Tecnoplast	1263459.17
6	Fundição Ferro Forte	1055798.55

Fonte: Elaborado pelos autores (2026).

A base de dados apresenta apenas seis fornecedores cadastrados, todos com volume de compras na mesma ordem de grandeza (entre R\$ 1.055.798,55 e R\$ 1.685.990,89), o que indica uma base de fornecimento relativamente equilibrada, sem a forte concentração observada no lado da receita (Indicador 3). O fornecedor "Motores Elétricos WEG-like" lidera o custo total acumulado, seguido por "Rolamentos & Cia" e "Vedações Selomax". A natureza dos fornecedores listados é compatível com motores elétricos, rolamentos, vedações, aços especiais, polímeros e fundição, na qual é compatível com a cadeia de suprimentos típica de um fabricante de bombas e válvulas industriais, reforçando a coerência entre os dados da base "erp" e o portfólio de produtos identificado na tabela "produto" (famílias de bombas, conjuntos pressurizadores e válvulas).

6.5 Indicador 5 - Ordens de produção por status

Figura 29 - Consulta de ordens de produção por status

Query		Query History	
1	SELECT	status, COUNT(*) AS total_ordens, SUM(qtd_planejada) AS qtd_total_planejada	
2	FROM	erp.ordem_producao	
3	GROUP BY	status	
4	ORDER BY	total_ordens DESC;	

Data Output		Messages	Notifications
Showing rows: 1 to 1	Page No:		
status	total_ordens	qtd_total_planejada	
character varying (15)	bigint	bigint	
1	CONCLUIDA	96	594566

Fonte: Elaborado pelos autores (2026).

De forma semelhante ao observado no Indicador 1, a consulta retornou um único status, "CONCLUIDA", totalizando 96 ordens de produção e 594.566 unidades planejadas. A ausência de ordens em status intermediário (como "em execução" ou "aberta") confirma que a base de dados representa um histórico de operações já finalizadas, adequado à análise retrospectiva de desempenho produtivo, como a aderência ao plano mestre, discutida no Indicador 6, mas insuficiente para indicadores de chão de fábrica em tempo real, função que, na arquitetura de sistemas de informação discutida na seção 2.4, seria mais adequadamente atendida pelo banco de dados não relacional "mes" (MES), analisado na seção posterior deste trabalho.

6.6 Indicador 6 - Aderência entre plano mestre de produção e previsão de demanda

Figura 30 - Consulta de aderência entre plano mestre e previsão de demanda

Query Query History	
1	SELECT pm.produto_id, p.nome, pm.ano_mes,
2	pm.qtd_planejada, pd.qtd_prevista,
3	(pm.qtd_planejada - pd.qtd_prevista) AS desvio
4	FROM erp.plano_mestre pm
5	JOIN erp.produto p ON p.id = pm.produto_id
6	LEFT JOIN erp.previsao_demanda pd
7	ON pd.produto_id = pm.produto_id AND pd.ano_mes = pm.ano_mes
8	ORDER BY pm.ano_mes, p.nome;

Data Output Messages Notifications	
Showing rows: 1 to 96	
produto_id smallint	nome character varying (80)
ano_mes character (7)	qtd_planejada integer
qtd_prevista integer	desvio integer
1	Bomba Centrífuga 1cv
2	Bomba Centrífuga 2cv
3	Bomba Submersa 3cv
4	Conjunto Booster Duplo
5	Conjunto Pressurizador
6	Válvula Esfera 1/2"
7	Válvula Esfera 3/4"
8	Válvula Gaveta 1"
9	Bomba Centrífuga 1cv
10	Bomba Centrífuga 2cv
11	Bomba Submersa 3cv
12	Conjunto Booster Duplo

Fonte: Elaborado pelos autores (2026).

A consulta retornou 96 registros, comparando, mês a mês, a quantidade planejada no plano mestre de produção (MPS) com a quantidade prevista de demanda para cada produto. Observa-se que os desvios variam tanto para valores positivos (planejamento acima da previsão, como a Bomba Centrífuga 2cv em 2025-01, com desvio de +297 unidades) quanto negativos (planejamento abaixo da previsão, como a Válvula Esfera 3/4" no mesmo mês, com desvio de -471 unidades), sem um padrão sistemático de superdimensionamento ou subdimensionamento entre os produtos.

Esse comportamento é coerente com a prática de planejamento agregado de produção discutida por Slack, Brandon-Jones e Johnston (2018), na qual o plano mestre busca equilibrar a previsão de demanda com restrições de capacidade produtiva e política de estoque, podendo gerar planejamento ligeiramente superior ou inferior à previsão pura, conforme a estratégia adotada (produção para estoque, produção contra pedido, nivelamento de capacidade, entre outras). A ausência de desvios sistemáticos e de grande magnitude indica razoável aderência entre o planejamento mestre e a previsão de demanda na base analisada.

6.7 Indicador 7 - Resultado por grupo de DRE

Figura 31 - Consulta de resultado por grupo de DRE

Query

Query History

1

2

3

4

5

6

```
SELECT gd.nome AS grupo_dre, SUM(lc.valor) AS valor_total
FROM erp.lancamento_contabil lc
JOIN erp.conta_contabil cc ON cc.id = lc.conta_id
JOIN erp.grupo_dre gd ON gd.codigo = cc.grupo_dre_codigo
GROUP BY gd.nome, gd.ordem
ORDER BY gd.ordem;
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 8

	grupo_dre character varying (60)	valor_total numeric
1	Receita Bruta	562314028.70
2	Deduções da Receita	-157478654.28
3	Custo dos Produtos Vendid...	-267698474.00
4	Despesas Comerciais	-29128199.67
5	Despesas Administrativas	-34589491.45
6	Depreciação e Amortização	-4560000.00
7	Resultado Financeiro	-2625069.43
8	Provisão IRPJ/CSLL	-22519607.55

Fonte: Elaborado pelos autores (2026).

A consulta consolida os lançamentos contábeis segundo a estrutura de Demonstração de Resultado do Exercício (DRE), permitindo reconstruir o resultado financeiro da organização a partir dos dados transacionais do banco "erp". Partindo de uma Receita Bruta de R\$ 562.314.028,70, e descontando sucessivamente as Deduções da Receita (R\$ 157.478.654,28), o Custo dos Produtos Vendidos (R\$ 267.698.474,00), as Despesas Comerciais (R\$ 29.128.199,67), as Despesas Administrativas (R\$ 34.589.491,45), a Depreciação e Amortização (R\$ 4.560.000,00), o Resultado Financeiro negativo (R\$ 2.625.069,43) e a Provisão de IRPJ/CSLL (R\$ 22.519.607,55), obtém-se um resultado líquido aproximado de R\$ 43.714.532,32, correspondente a uma margem líquida de aproximadamente 7,8% sobre a receita bruta.

Esse resultado evidencia que o Custo dos Produtos Vendidos representa a maior parcela de redução da receita (47,6% da receita bruta), seguido pelas Deduções da Receita (28,0%), reforçando a relevância da gestão de custos de produção, diretamente associada às tabelas de estrutura de produto, roteiro e movimentação de estoque analisadas neste trabalho, como fator determinante da lucratividade da organização.

6.8 Síntese dos indicadores

Tabela 2 - Síntese dos indicadores extraídos do banco de dados ERP

Indicador	Resultado Principal	Implicação Gerencial
Pedidos por status	152 pedidos, todos faturados	Histórico consolidado de vendas
Estoque versus segurança	Sem rupturas; segurança de 50%	Estoque estável e política simplificada
Faturamento por cliente	Concentração nos três maiores clientes	Dependência de poucos clientes
Compras por fornecedor	Seis fornecedores com custos equilibrados	Cadeia de suprimentos diversificada
Ordens de produção	96 ordens, todas concluídas	Histórico produtivo consolidado
Plano mestre × previsão	Pequenos desvios observados	Planejamento aderente à demanda
Resultado por grupo de DRE	Margem líquida de aproximadamente 7,8%; CPV predominante	Controle de custos como prioridade

Fonte: Elaborado pelos autores (2026).

7. Consultas e indicadores extraídos do banco "mes"

Diferentemente da base "erp", cujos indicadores estão predominantemente associados aos processos administrativos, comerciais e de planejamento, os indicadores extraídos do banco de dados "mes" estão relacionados ao acompanhamento da execução das operações produtivas e ao monitoramento de eventos ocorridos em chão de fábrica. As coleções analisadas armazenam informações referentes ao estado das máquinas, apontamentos de produção, manutenções e inspeções de qualidade, possibilitando a obtenção de métricas voltadas à avaliação do desempenho operacional dos recursos produtivos. Nesse contexto, as consultas apresentadas nas subseções seguintes têm como objetivo demonstrar como os dados operacionais registrados pelo MES podem ser transformados em informações gerenciais relevantes para atividades de monitoramento, controle e suporte à tomada de decisão no ambiente industrial.

7.1 Indicador 8 -Disponibilidade operacional das máquinas

Figura 32 - Consulta de tempo planejado, operando e parado

```
>_MONGOSH
> use mes
< already on db mes
> db.apontamentos_producao.aggregate([
  {
    $group: {
      _id: null,
      tempo_planejado_total: { $sum: "$tempo_planejado_min" },
      tempo_operando_total: { $sum: "$tempo_operando_min" },
      tempo_parado_total: { $sum: "$tempo_parado_min" }
    }
  }
])
< {
  _id: null,
  tempo_planejado_total: 939600,
  tempo_operando_total: 842809,
  tempo_parado_total: 96791
}
mes>
```

Fonte: Elaborado pelos autores (2026).

A consulta totalizou 939.600 minutos planejados de produção, dos quais 842.809 minutos (89,7%) corresponderam a tempo efetivamente operando e 96.791 minutos (10,3%) corresponderam a tempo parado. Esse indicador representa a disponibilidade operacional do parque de máquinas, conceito central no monitoramento de processos produtivos em tempo real discutido por Kagermann, Wahlster e Helbig (2013) no contexto da Indústria 4.0, no qual sensores e sistemas MES capturam o estado da máquina em tempo real, permitindo o cálculo de indicadores de eficiência operacional global.

Uma disponibilidade de 89,7% é considerada um valor relativamente elevado para ambientes industriais, sugerindo um parque de máquinas com bom desempenho operacional, ainda que o detalhamento dos motivos de parada (Indicador 9) seja necessário para identificar oportunidades de melhoria.

7.2 Indicador 9 - Distribuição do tempo de parada por motivo

Figura 33 - Consulta de motivos de parada

```
> db.apontamentos_producao.aggregate([
  { $unwind: "$paradas" },
  {
    $group: {
      _id: "$paradas.motivo",
      total_ocorrencias: { $sum: 1 },
      tempo_total_min: { $sum: "$paradas.duracao_min" }
    }
  },
  { $sort: { tempo_total_min: -1 } }
])
< {
  _id: 'Ajuste de processo',
  total_ocorrencias: 884,
  tempo_total_min: 18296
}
{
  _id: 'Falta de operador',
  total_ocorrencias: 833,
  tempo_total_min: 16578
}
{
  _id: 'Falta de material',
  total_ocorrencias: 821,
  tempo_total_min: 16235
}
{
  _id: 'Setup/Troca de ferramenta',
  total_ocorrencias: 807,
  tempo_total_min: 15520
}
{
  _id: 'Pequenas paradas',
  total_ocorrencias: 781,
  tempo_total_min: 15244
}
{
  _id: 'Manutenção corretiva (quebra)',
  total_ocorrencias: 159,
  tempo_total_min: 14918
}
```

Fonte: Elaborado pelos autores (2026).

A Figura 33 apresenta os seis motivos de parada identificados, ordenados pelo tempo total acumulado. A soma do tempo de todos os motivos (96.791 minutos) corresponde exatamente ao tempo parado total apurado no Indicador 8, confirmando a consistência entre as duas consultas e validando o uso do campo "paradas" (documento embutido) como fonte completa de detalhamento das interrupções de produção. Observa-se que "Ajuste de processo" é o motivo com maior tempo acumulado (18.296 minutos, 18,9% do tempo total parado), seguido por "Falta de operador" (16.578 min) e "Falta de material" (16.235 min). Chama atenção que, embora a "Manutenção corretiva (quebra)" apresente o maior tempo médio por ocorrência ($14.918 \div 159 \approx 93,8$ minutos por evento), seu impacto no tempo total agregado (14.918 min) é o menor entre os seis motivos, justamente por ocorrer com menor frequência. Já motivos como "Ajuste de processo" e "Setup/Troca de ferramenta", apesar de apresentarem menor duração média por evento, acumulam maior impacto total em razão da alta frequência de ocorrência (884 e 807 eventos, respectivamente). Essa distinção é relevante para a priorização de ações de melhoria contínua: enquanto a redução de quebras exige investimento em manutenção preventiva (relacionado ao Indicador 12), a redução de paradas por ajuste de processo e setup está mais associada a métodos de melhoria como SMED (Single-Minute Exchange of Die) e padronização de processos (Johnston, 2018).

7.3 Indicador 10 – Taxa de refugo de produção

Figura 34 – Consulta de peças totais, boas e refugadas

```
> db.apontamentos_producao.aggregate([
  {
    $group: {
      _id: null,
      pecas_total: { $sum: "$pecas_total" },
      pecas_boas: { $sum: "$pecas_boas" },
      pecas_refugo: { $sum: "$pecas_refugo" }
    }
  }
])
< {
  _id: null,
  pecas_total: 606462,
  pecas_boas: 596500,
  pecas_refugo: 9962
}
```

Fonte: Elaborado pelos autores (2026).

Do total de 606.462 peças produzidas no período analisado, 596.500 (98,36%) foram classificadas como boas e 9.962 (1,64%) foram refugadas. Uma taxa de refugo de 1,64% é considerada baixa para a maioria dos contextos industriais, sugerindo processos produtivos relativamente estáveis. Esse indicador, combinado ao Indicador 13 (taxa de aprovação na inspeção de qualidade), permite uma visão consolidada da qualidade do processo produtivo registrado na base "mes".

7.4 Indicador 11 -Parque de máquinas e status operacional

Figura 35 - Consulta do cadastro de máquinas

```
> db.maquinas.find({}, { codigo: 1, nome: 1, status: 1, fabricante: 1 })
< {
  _id: ObjectId('6a1340efcac062957a9e01eb'),
  codigo: 'TOR-01',
  nome: 'Torno CNC 01',
  fabricante: 'Romi',
  status: 'ATIVA'
}
{
  _id: ObjectId('6a1340efcac062957a9e01ec'),
  codigo: 'TOR-02',
  nome: 'Torno CNC 02',
  fabricante: 'Romi',
  status: 'ATIVA'
}
{
  _id: ObjectId('6a1340efcac062957a9e01ed'),
  codigo: 'INJ-01',
  nome: 'Injetora 01',
  fabricante: 'Haitian',
  status: 'ATIVA'
}
{
  _id: ObjectId('6a1340efcac062957a9e01ee'),
  codigo: 'MON-01',
  nome: 'Linha de Montagem 01',
  fabricante: 'Interna',
  status: 'ATIVA'
}
```

Fonte: Elaborado pelos autores (2026).

O parque de máquinas registrado na base "mes" é composto por quatro recursos produtivos: dois tornos CNC (TOR-01 e TOR-02, fabricante Romi), uma injetora (INJ-01, fabricante Haitian) e uma linha de montagem (MON-01, fabricante Interna), todos com status "ATIVA" no momento da consulta. A ausência de máquinas inativas ou em manutenção prolongada no cadastro é coerente com a elevada disponibilidade operacional verificada no Indicador 8.

7.5 Indicador 12 - Custo de manutenção corretiva versus preventiva

Figura 36 - Consulta de custo e tempo de manutenção por tipo

```
> db.manutencoes.aggregate([
  {
    $group: {
      _id: "$tipo",
      total_ocorrencias: { $sum: 1 },
      custo_total: { $sum: "$custo" },
      tempo_total_min: { $sum: "$duracao_min" }
    }
  },
  { $sort: { custo_total: -1 } }
])
< {
  _id: 'CORRETIVA',
  total_ocorrencias: 159,
  custo_total: 846415.92,
  tempo_total_min: 14918
}
{
  _id: 'PREVENTIVA',
  total_ocorrencias: 48,
  custo_total: 170136.31,
  tempo_total_min: 5490
}
```

Fonte: Elaborado pelos autores (2026).

Das 207 manutenções registradas, 159 (76,8%) foram corretivas e 48 (23,2%) foram preventivas. O custo total de manutenção corretiva (R\$ 846.415,92) representou 83,3% do custo total de manutenção (R\$ 1.016.552,23), substancialmente superior à sua proporção em número de ocorrências. O custo médio por evento reforça essa disparidade: cada manutenção corretiva custou, em média, R\$ 5.323,37, enquanto cada manutenção preventiva custou, em média, R\$ 3.544,51, ou seja, a manutenção corretiva é, em média, 50,2% mais cara por evento do que a preventiva. Esse resultado é consistente com a literatura de gestão de manutenção, segundo a qual falhas não planejadas tendem a gerar custos adicionais relacionados a paradas não programadas, danos secundários a componentes e urgência no reparo, reforçando o argumento em favor da ampliação de programas de manutenção preventiva como estratégia de redução de custos no médio prazo (Johnston, 2018).

7.6 Indicador 13 - Taxa de aprovação na inspeção de qualidade

Figura 37 - Consulta de aprovação nas inspeções de qualidade

```
> db.inspecoes_qualidade.aggregate([
  {
    $group: {
      _id: "$aprovado",
      total: { $sum: 1 }
    }
  }
])
< {
  _id: false,
  total: 6
}
{
  _id: true,
  total: 186
}
```

Fonte: Elaborado pelos autores (2026).

Das 192 inspeções de qualidade registradas, 186 (96,9%) resultaram em aprovação e 6 (3,1%) resultaram em reprovação. Essa taxa de aprovação, combinada à baixa taxa de refugo verificada no Indicador 10 (1,64%), reforça a consistência interna da base de dados quanto à qualidade do processo produtivo, e demonstra como as coleções "apontamentos_producao" e "inspecoes_qualidade" capturam, de perspectivas complementares (produção e controle de qualidade), um mesmo processo produtivo estável.

7.7 Síntese dos indicadores

Tabela 3 - Síntese dos indicadores extraídos do banco de dados mes

Indicador	Resultado principal
Disponibilidade operacional	89,7% do tempo planejado operando
Motivos de parada	Ajuste de processo é o principal motivo (18,9%)
Taxa de refugo	1,64% das peças produzidas
Status do parque de máquinas	4 máquinas, 100% ativas
Custo de manutenção corretiva x preventiva	Corretiva custa 50,2% mais por evento
Taxa de aprovação na inspeção de qualidade	96,9% de aprovação

Fonte: Elaborado pelos autores (2026).

8 - Considerações Finais

A realização das atividades práticas permitiu consolidar, de forma aplicada, os conceitos abordados ao longo do trabalho acerca dos sistemas de banco de dados relacionais e não relacionais, evidenciando suas diferenças estruturais, suas particularidades de modelagem e suas possibilidades de utilização em ambientes industriais. A execução do Procedimento Operacional Padrão (POP) possibilitou não apenas o estabelecimento das conexões com os bancos de dados "erp" e "mes", mas também a exploração efetiva de suas estruturas, a identificação de seus relacionamentos e a extração de informações relevantes para a análise de processos organizacionais.

No banco de dados relacional "erp", verificou-se uma estrutura composta por entidades fortemente integradas por meio de chaves primárias e estrangeiras, apresentando elevado grau de organização e conformidade com os princípios de normalização. O mapeamento realizado permitiu identificar relacionamentos de diferentes cardinalidades, relações de agregação e composição e a aplicação prática das Primeira, Segunda e Terceira Formas Normais. Esses aspectos evidenciam uma modelagem orientada à integridade, à consistência e à redução de redundâncias, características fundamentais para sistemas de gestão empresarial que demandam elevado nível de confiabilidade das informações transacionais.

Por sua vez, a análise do banco de dados não relacional "mes" evidenciou uma abordagem de modelagem significativamente distinta, baseada em documentos, coleções e estruturas flexíveis de armazenamento. A utilização de documentos embutidos e de referências por chaves de negócio demonstrou uma modelagem adequada ao registro de eventos operacionais, apontamentos de produção, manutenções e inspeções de qualidade, contextos nos quais a flexibilidade estrutural e o desempenho de leitura constituem requisitos importantes. Observou-se ainda que determinadas práticas, como a desnormalização e a duplicação controlada de informações, embora representassem inconsistências em um modelo relacional tradicional, mostraram-se coerentes com os objetivos de desempenho e rastreabilidade inerentes ao modelo documental.

A análise integrada das duas bases permitiu observar, na prática, a complementaridade entre os modelos de persistência de dados. Enquanto o banco "erp" concentra informações administrativas, comerciais e de planejamento, o banco "mes" registra informações relacionadas à execução das operações produtivas e aos eventos ocorridos em chão de fábrica.

A correspondência entre elementos presentes em ambas as bases, como ordens de produção e identificadores de produtos, demonstrou a existência de um fluxo de informações que conecta o planejamento, a execução e o acompanhamento das operações, reproduzindo uma dinâmica frequentemente observada em sistemas industriais contemporâneos.

As consultas realizadas sobre a base "erp" evidenciaram também o potencial analítico dos bancos de dados para geração de informações gerenciais. Os indicadores extraídos permitiram identificar um cenário sem ocorrência de ruptura de estoque, padrões de parametrização dos estoques de segurança e elevada concentração de faturamento em um número reduzido de clientes. Embora a base disponibilizada represente um recorte consolidado e não permita avaliar determinados indicadores de desempenho operacional em tempo real, os resultados obtidos demonstraram como dados estruturados podem ser transformados em informações capazes de apoiar atividades de monitoramento, análise e tomada de decisão.

De maneira geral, os resultados alcançados permitiram verificar que os bancos de dados constituem elementos centrais para o funcionamento dos sistemas de informação organizacionais e para a gestão dos processos produtivos e administrativos. A aplicação prática desenvolvida ao longo deste trabalho possibilitou a transição dos conceitos teóricos para um contexto de utilização real, proporcionando uma compreensão mais aprofundada sobre modelagem de dados, integração entre sistemas, extração de informações e utilização de indicadores gerenciais. Sob a perspectiva da Engenharia de Produção, a experiência desenvolvida evidencia que a capacidade de compreender, estruturar e analisar dados provenientes de diferentes fontes tornou-se uma competência essencial para o desenvolvimento de soluções voltadas ao planejamento, ao controle e à melhoria contínua dos sistemas produtivos.

REFERÊNCIAS

- BANKER, Kyle. MongoDB in Action. New York: Manning Publications, 2011.
- BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. UML: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2005.
- BREWER, Eric. CAP twelve years later: how the "rules" have changed. Computer, Los Alamitos, v. 45, n. 2, p. 23-29, 2012.
- CHIAVENATO, Idalberto. Introdução à teoria geral da administração. 9. ed. Barueri: Manole, 2014.
- CHODOROW, Kristina. MongoDB: The Definitive Guide. 3. ed. Sebastopol: O'Reilly Media, 2019.
- CODD, Edgar F. A relational model of data for large shared data banks. Communications of the ACM, New York, v. 13, n. 6, p. 377-387, 1970.

- CONNOLLY, Thomas; BEGG, Carolyn. Database Systems: A Practical Approach to Design, Implementation and Management. 6. ed. Boston: Pearson, 2015.
- CORONEL, Carlos; MORRIS, Steven. Database Systems: Design, Implementation and Management. 12. ed. Boston: Cengage Learning, 2016.
- DATE, C. J. Introdução a Sistemas de Bancos de Dados. 8. ed. Rio de Janeiro: Elsevier, 2004.
- ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados. 7. ed. São Paulo: Pearson, 2018.
- FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.
- FOWLER, Martin; SADALAGE, Pramod J. NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Boston: Addison-Wesley, 2012.
- FREEMAN, Eric; ROBSON, Elisabeth. Head First Design Patterns. Sebastopol: O'Reilly Media, 2004.
- KAGERMANN, Henning; WAHLSTER, Wolfgang; HELBIG, Johannes. Recommendations for Implementing the Strategic Initiative INDUSTRIE 4.0. Frankfurt: National Academy of Science and Engineering, 2013.
- LAUDON, Kenneth C.; LAUDON, Jane P. Sistemas de Informação Gerenciais. 11. ed. São Paulo: Pearson Education do Brasil, 2014.
- MCKINNEY, Wes. Python for Data Analysis. 3. ed. Sebastopol: O'Reilly Media, 2022.
- MONGODB INC. MongoDB Documentation. Disponível em: <https://www.mongodb.com/docs/>. Acesso em: 2 maio 2026.
- O'BRIEN, James A.; MARAKAS, George M. Administração de Sistemas de Informação. 15. ed. Porto Alegre: AMGH, 2013.
- POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 2 maio 2026.
- PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 9. ed. Porto Alegre: AMGH, 2020.
- PSYCOPG DEVELOPMENT TEAM. Psycopg Documentation. Disponível em: <https://www.psycopg.org/docs/>. Acesso em: 2 maio 2026.
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de Banco de Dados. 7. ed. Rio de Janeiro: Elsevier, 2020.
- SLACK, Nigel; BRANDON-JONES, Alistair; JOHNSTON, Robert. Administração da Produção. 8. ed. São Paulo: Atlas, 2018.
- SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.
- STAIR, Ralph M.; REYNOLDS, George W. Princípios de Sistemas de Informação. 11. ed. São Paulo: Cengage Learning, 2015.